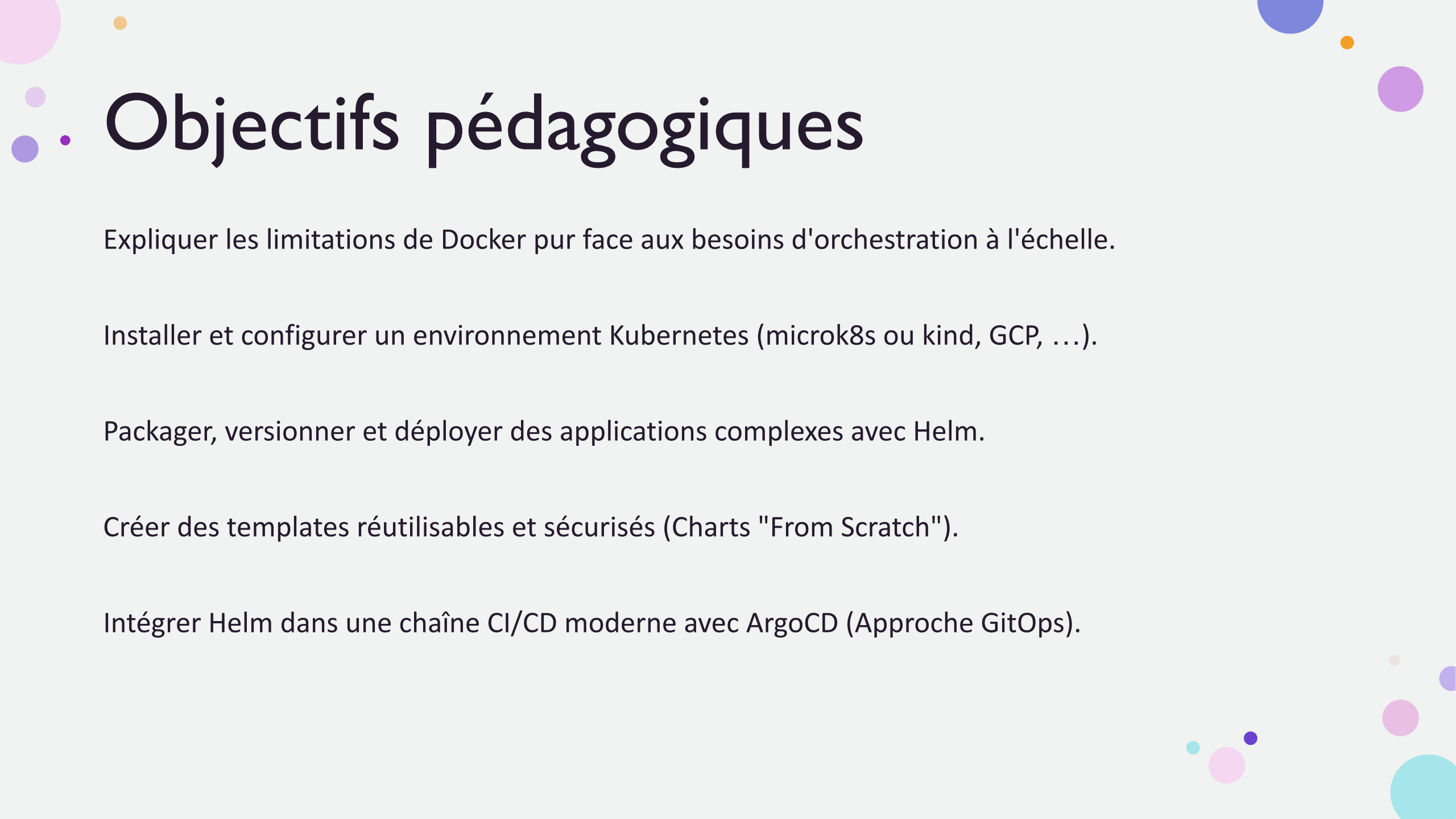




Formation HELM

Ali Koudri

ali.koudri@janus-academy.fr

The slide features a light blue background with several decorative circles of various colors (pink, purple, blue, orange, teal) scattered in the top-left and top-right corners. The main title is a large, dark purple, sans-serif font.

Objectifs pédagogiques

Expliquer les limitations de Docker pur face aux besoins d'orchestration à l'échelle.

Installer et configurer un environnement Kubernetes (microk8s ou kind, GCP, ...).

Packager, versionner et déployer des applications complexes avec Helm.

Créer des templates réutilisables et sécurisés (Charts "From Scratch").

Intégrer Helm dans une chaîne CI/CD moderne avec ArgoCD (Approche GitOps).

Agenda

- Rappels sur Kubernetes / Kustomize
- Présentation Générale de Helm
- Écosystème Cloud Native
- Prise en main et commandes fondamentales
- Personnalisation des déploiements
- Développements de Charts “From Scratch”
- Templating Avancé
- Qualité et Tests
- Intégration avec Kustomize
- Intégration avec ArgoCD

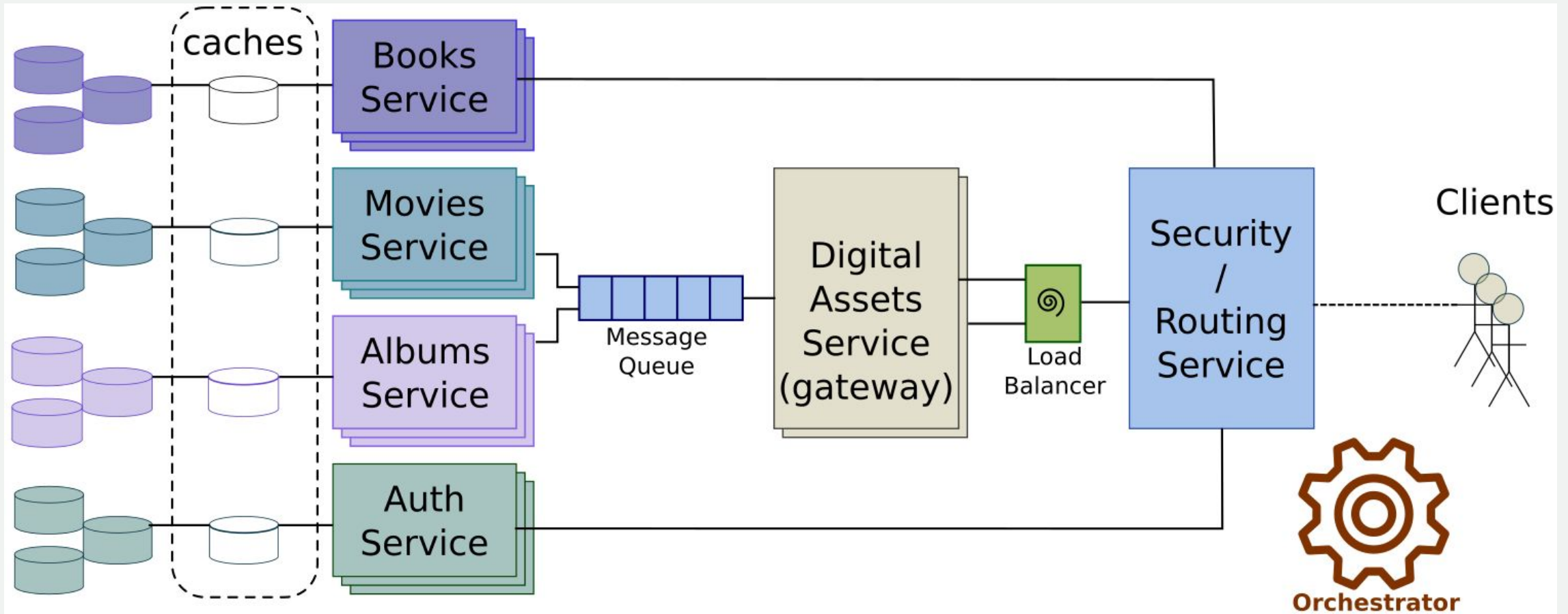
Repo Git Labs : <https://github.com/akoudri/devops-news-lab.git>

The slide features a light gray background with decorative elements in the corners. The top-left corner has a cluster of pink, purple, and orange circles. The top-right corner has blue, purple, and orange circles. The bottom-right corner has a cluster of pink, purple, and teal circles.

Rappels K8S - Partie I/4

Principes Généraux

Architecture en micro-services



- **Question:** Qu'attend-on d'un déploiement cloud ? Quels sont les problèmes ?

K8S - Motivations

Orchestration multi-hôtes

Docker gère les conteneurs sur un seul hôte ou quelques hôtes, mais il ne fournit pas nativement une orchestration avancée sur un cluster complet. Kubernetes permet de gérer, déployer et faire évoluer des applications sur un cluster de nombreux nœuds, assurant ainsi la haute disponibilité et la répartition de charge automatique.

Mise à l'échelle automatique

Docker ne propose pas de mécanisme natif pour augmenter ou réduire dynamiquement le nombre de conteneurs selon la charge. Kubernetes automatise la mise à l'échelle horizontale et verticale des applications en fonction de la demande.

Haute disponibilité et tolérance aux pannes

Docker seul ne redémarre pas automatiquement les conteneurs ou ne les déplace pas en cas de panne d'un nœud. Kubernetes surveille l'état des applications et des nœuds, redémarre ou remplace automatiquement les conteneurs défectueux, et garantit la disponibilité du service.

K8S - Motivations

Équilibrage de charge intégré

Docker ne gère pas nativement la répartition du trafic réseau entre plusieurs conteneurs. Kubernetes fournit un équilibrage de charge automatique pour répartir le trafic entre les instances de service.

Déploiements et mises à jour automatisés

Docker ne propose pas de stratégies sophistiquées pour les mises à jour continues ou les rollbacks. Kubernetes permet des déploiements progressifs, des mises à jour sans interruption de service et des retours arrière automatisés en cas d'échec.

Gestion du stockage persistant

Docker ne gère pas nativement la persistance des données lors du redémarrage ou du déplacement des conteneurs. Kubernetes orchestre le stockage persistant et l'attache dynamiquement aux conteneurs où qu'ils soient exécutés.

Découverte de services

Docker nécessite une configuration manuelle pour que les conteneurs se découvrent entre eux. Kubernetes fournit des mécanismes de découverte de services intégrés, facilitant la communication entre composants applicatifs.

Helm - Motivations

L'explosion du YAML

Déployer une application Kubernetes réelle nécessite des dizaines de fichiers YAML. Sans outil de packaging, ces fichiers se dupliquent pour chaque environnement, deviennent incohérents, et personne ne sait quelle version est réellement en production.

Pas de gestion de version native

Kubernetes déploie des ressources individuellement mais n'a aucune notion d'application versionnée. Helm regroupe toutes les ressources d'une application dans une Release horodatée et réversible, exactement comme un commit Git.

La rigidité du YAML statique

Les manifests Kubernetes bruts ne supportent ni variables, ni conditions, ni boucles. Déployer la même application en Dev avec 1 réplica et en Prod avec 5 impose de maintenir deux fichiers quasi-identiques. Helm résout ce problème avec son moteur de templating Go.

Helm - Motivations

La distribution impossible

Partager une application Kubernetes avec une autre équipe revient à leur envoyer un dossier de fichiers YAML non documentés. Helm package l'application, ses dépendances et sa configuration en une archive versionnée publiable sur un registre, comme une image Docker.

Le rollback manuel, une opération risquée

Sans Helm, revenir à une version précédente d'une application nécessite de retrouver et réappliquer manuellement les anciens manifests. Helm conserve l'historique complet de chaque Release et permet un rollback atomique en une seule commande, en quelques secondes.

L'atomicité absente

Kubernetes applique les ressources une par une. Si un Deployment échoue après que son Service a été créé, le cluster se retrouve dans un état incohérent. Helm traite l'installation comme une transaction : en cas d'échec, il peut automatiquement revenir à l'état précédent.

Installation de K8S

Pour faire des tests localement, il est recommandé d'utiliser microk8s ou kind

- <https://ubuntu.com/tutorials/install-a-local-kubernetes-microk8s#1-overview>
- <https://kind.sigs.k8s.io>

Vous pouvez également passer par un fournisseur

- Amazon EKS: <https://aws.amazon.com/eks/>
- Google GKE: <https://cloud.google.com/kubernetes-engine?hl=fr>
- Azure AKS: <https://learn.microsoft.com/fr-fr/azure/aks/>

Ou mettre en place votre propre stack avec

- OpenShift
- OpenStack

Dans le cas où vous ne pouvez rien installer

- <https://labs.play-k8s.com/>
- <https://killercode.com/>

Installation locale avec microk8s

`sudo snap install --classic microk8s`

Pod to pod and pod-to-pod communication:

- *`sudo ufw allow in on cni0 && sudo ufw allow out on cni0`*
- *`sudo ufw default allow routed`*

Addons:

- *`microk8s enable dns`*
- *`microk8s enable dashboard`*
- *`microk8s enable hostpath-storage`*
- *`microk8s enable metrics-server`*

Accès au dashboard:

- Génération d'un token de sécurité:
 - *`token=$(kubectl -n kube-system get secrets | grep token | cut -d " " -f1)`*
 - *`microk8s kubectl -n kube-system describe secret $token`*

Installation locale avec Kind

Présentation

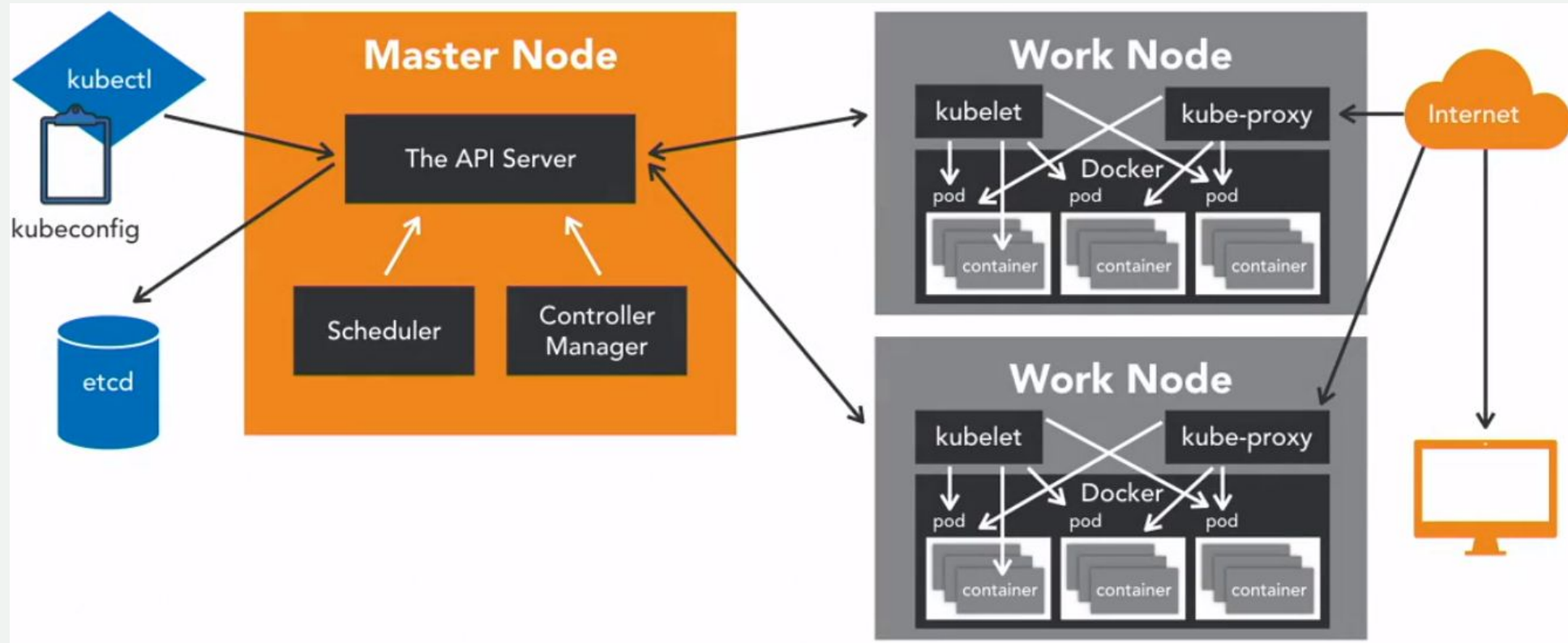
- **Concept** : Un outil pour exécuter des clusters Kubernetes locaux en utilisant des conteneurs Docker comme "nœuds".
- **Usage** : Principalement conçu pour tester Kubernetes lui-même, mais très utilisé pour le développement local et l'intégration continue (CI).
- **Avantage majeur** : Permet de simuler très facilement des clusters multi-nœuds sur une seule machine (contrairement à Minikube ou MicroK8s qui sont souvent mono-nœud par défaut).

Commandes essentielles

- **Créer un cluster** : `kind create cluster --name mon-cluster`
- **Déployer une config multi-nœuds** : `kind create cluster --config kind-config.yaml`
- **Supprimer un cluster** : `kind delete cluster --name mon-cluster`

Cf. fichier scripts/kind-[create-cluster.sh](#) pour l'installation

Architecture - Vue d'ensemble



Source: <https://amdocs-1.gitbook.io/kubernetes-handbook>

API Server

Point d'entrée central du cluster

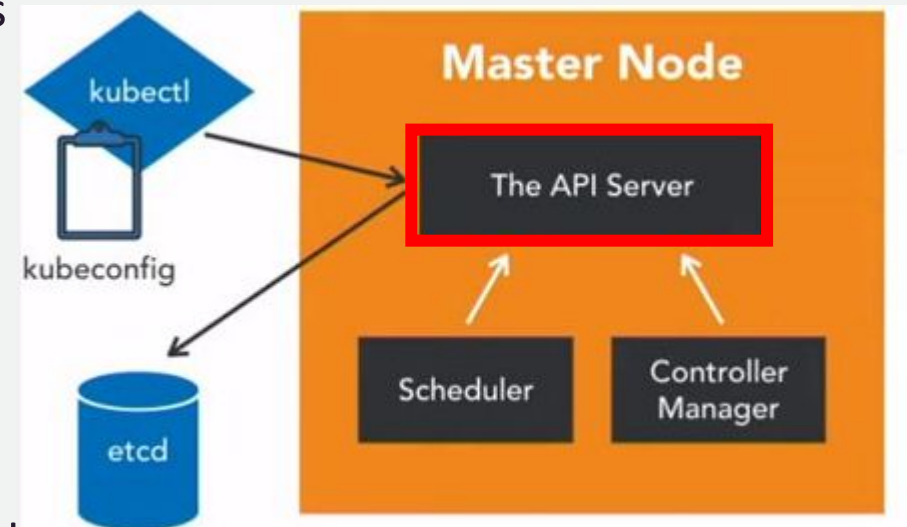
- L'API Server est le composant central du plan de contrôle Kubernetes.
- Il reçoit toutes les requêtes (création, modification, suppression, interrogation) concernant l'état du cluster, que ce soit depuis des utilisateurs, des outils comme kubectl, ou d'autres composants internes.

Interface de communication unique

- Il expose une API HTTP/REST qui permet aux utilisateurs, aux composants du cluster et aux systèmes externes de communiquer avec Kubernetes.
- Toutes les interactions passent par lui, ce qui garantit la cohérence et la sécurité des opérations.

Validation, authentification et autorisation

- Chaque requête reçue est authentifiée, validée et autorisée avant d'être traitée.
- L'API Server s'assure que seules les opérations valides et autorisées sont exécutées sur le cluster.



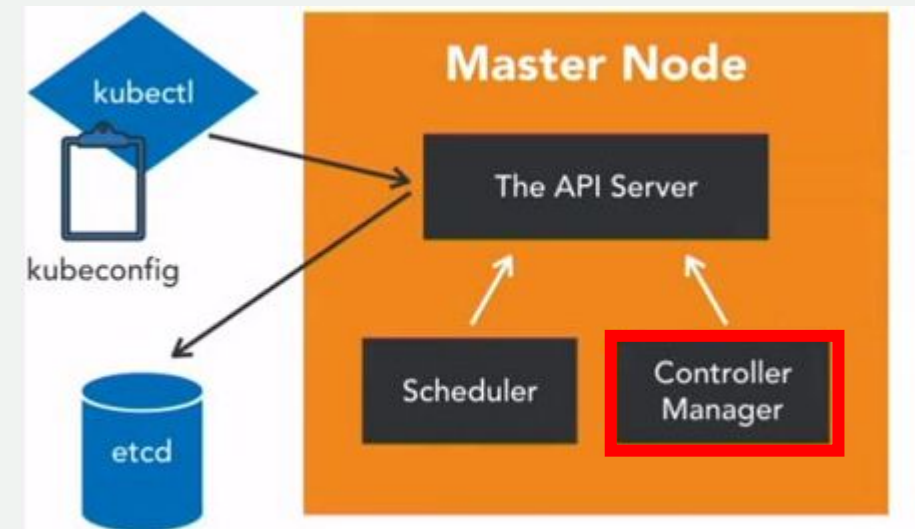
Controller Manager

Rôle central d'automatisation

Le Controller Manager est un composant clé du plan de contrôle de Kubernetes. Il orchestre l'automatisation du cluster en exécutant plusieurs "contrôleurs", qui sont des boucles logicielles surveillant en continu l'état du cluster pour le rapprocher de l'état désiré défini par l'utilisateur.

Réconciliation de l'état

Chaque contrôleur compare l'état réel du cluster à l'état attendu (défini dans les manifestes YAML) : s'il détecte un écart (par exemple, un pod supprimé alors qu'il doit y avoir 3 réplicas), il agit pour corriger la situation (ici, en recréant un pod).



Scheduler

Rôle principal

Le scheduler est responsable d'assigner chaque Pod nouvellement créé (et non encore planifié) à un nœud du cluster où il pourra être exécuté.

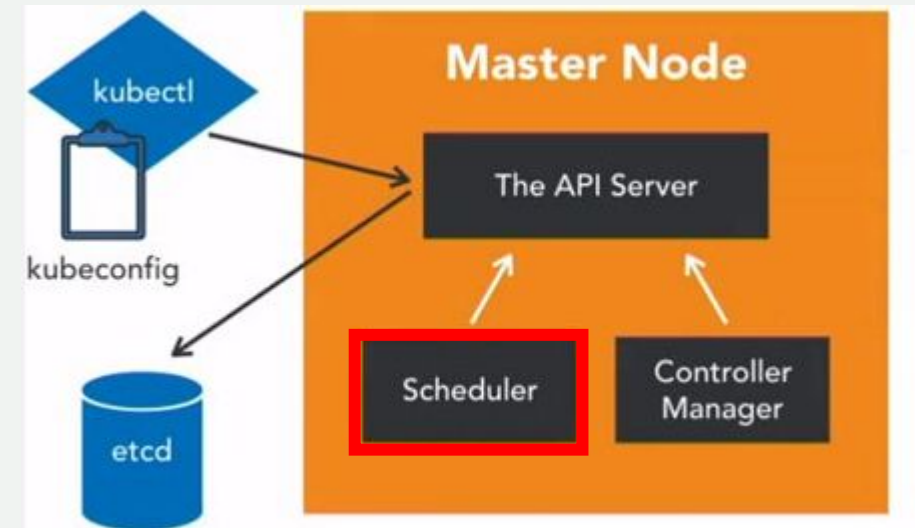
Processus de planification

Pour chaque Pod à planifier, le scheduler suit deux grandes étapes :

- **Filtrage** : il identifie la liste des nœuds « faisables » qui répondent aux contraintes du Pod (ressources disponibles, labels, affinités, taints/tolerations, etc.).
- **Scoring** : il attribue un score à chaque nœud filtré selon différents critères (charge, proximité des données, affinités, etc.), puis sélectionne le nœud ayant le score le plus élevé pour y placer le Pod.

Décision et binding

Une fois le meilleur nœud choisi, le scheduler notifie l'API Server de cette décision : c'est l'opération de binding. Le kubelet du nœud sélectionné prend alors le relais pour lancer le Pod.



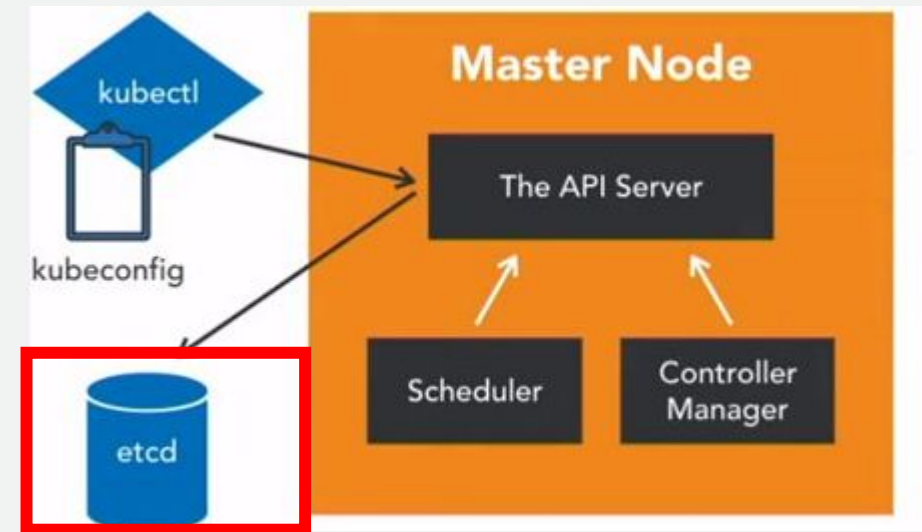
ETCD

Stockage de l'état du cluster

- **Source unique de vérité** : Etcd conserve toutes les données critiques du cluster, notamment les configurations, l'état des ressources (Pods, Services, Deployments), les secrets et les métadonnées.
- **Synchronisation** : Kubernetes utilise la fonction watch d'etcd pour détecter les écarts entre l'état réel et l'état désiré du cluster, déclenchant des actions correctives (ex. : redémarrage de Pods défectueux).

Haute disponibilité et cohérence

- **Consensus via Raft** : Etcd utilise le protocole Raft pour maintenir la cohérence des données entre les nœuds, même en cas de panne. Un leader est élu pour gérer les écritures, et les données sont répliquées sur un nombre impair de nœuds pour éviter les pertes.
- **Tolérance aux pannes** : En cas de défaillance d'un nœud, etcd garantit la continuité du service grâce à son mécanisme d'élection automatique.



Kubectl

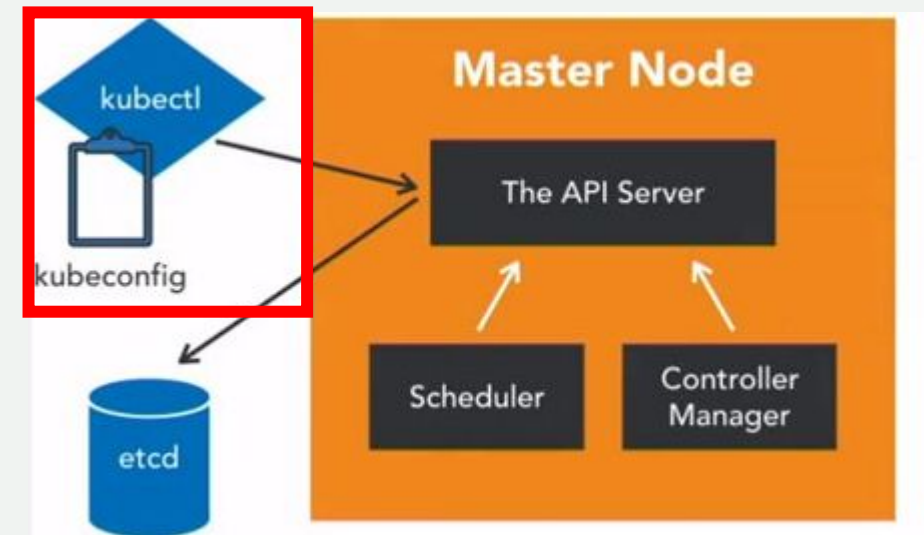
Outil en ligne de commande officiel de Kubernetes

Kubectl est l'interface principale pour administrer, déployer, surveiller et diagnostiquer les ressources d'un cluster Kubernetes directement depuis le terminal.

Communication avec l'API Server

Toutes les commandes envoyées via kubectl transitent par l'API Server de Kubernetes, ce qui permet d'effectuer toutes les opérations (création, lecture, modification, suppression) sur les objets du cluster.

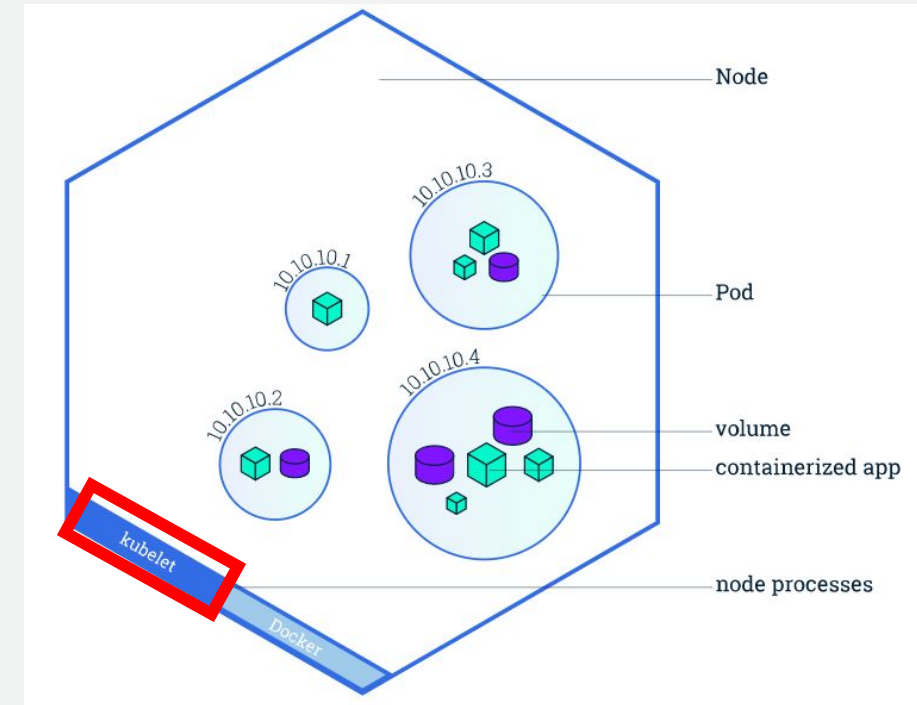
Question: Pourquoi toutes les communications doivent passer par l'API Server ?



Kubelet

Kubelet est le **chef d'orchestre local** sur chaque nœud Kubernetes : il applique les ordres du control plane, supervise l'exécution des Pods, surveille leur état et garantit l'automatisation et la résilience des applications conteneurisées du cluster

Il veille à ce que les Pods (et donc les conteneurs) soient créés, démarrés, maintenus et surveillés selon les spécifications reçues du plan de contrôle (control plane) via l'API Server.



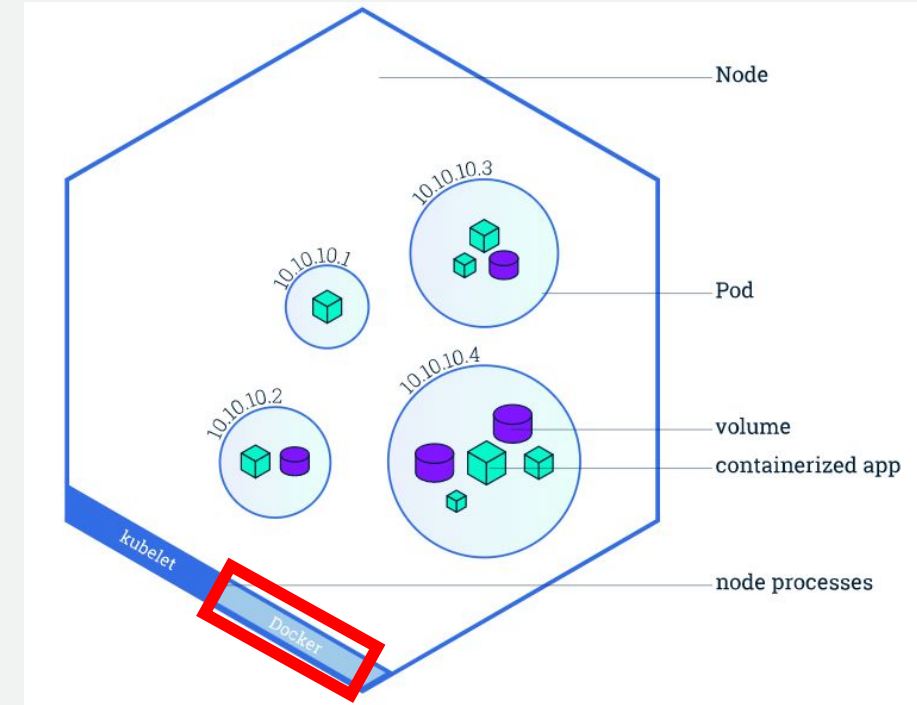
Kube-proxy

kube-proxy est un **composant réseau** essentiel qui s'exécute sur chaque nœud (worker node) d'un cluster Kubernetes.

Il agit comme un **proxy réseau** et un **load balancer** : il met en place les règles de routage nécessaires pour que le trafic réseau à destination des services Kubernetes soit correctement redirigé vers les Pods correspondants.

kube-proxy est le **chef d'orchestre réseau** de Kubernetes :

- Il assure la connectivité entre les services et les Pods,
- Met en place les règles réseau pour le routage et l'équilibrage de charge,
- S'adapte dynamiquement aux changements dans le cluster,
- Garantit l'accessibilité et la résilience des applications, aussi bien en interne qu'en externe



Pod

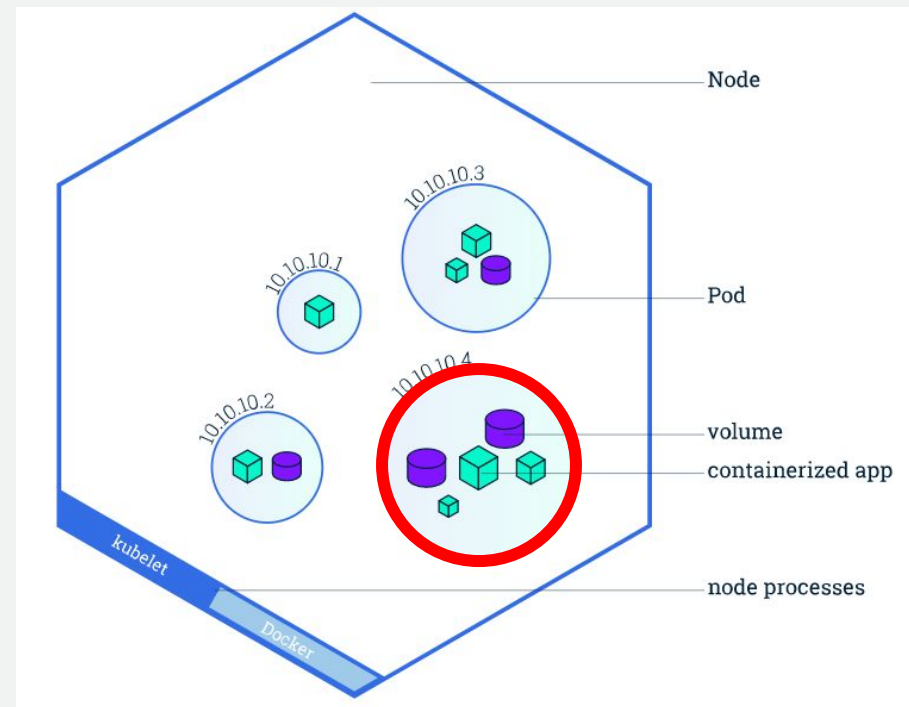
Le Pod est l'**unité minimale d'exécution** dans Kubernetes : il encapsule un ou plusieurs conteneurs qui partagent réseau et stockage.

Il représente un ou plusieurs conteneurs (généralement un seul) qui partagent le même espace réseau, le même stockage et le même cycle de vie.

Les conteneurs d'un même Pod sont toujours déployés ensemble sur le même nœud et peuvent communiquer facilement entre eux via localhost.

Il facilite la gestion, le déploiement et l'automatisation des applications conteneurisées dans le cluster.

Les objets de plus haut niveau (Deployment, ReplicaSet, etc.) gèrent en pratique des groupes de Pods pour assurer la scalabilité et la résilience.



Anatomie d'une commande kubectl

kubectl [commande] [TYPE] [NOM] [options]

- **commande** : l'action à effectuer (ex : get, describe, apply, delete, etc.)
- **TYPE** : le type de ressource ciblée (ex : pod, deployment, service, etc.), au singulier, pluriel ou abréviation (po pour pod, svc pour service...)
- **NOM** : le nom de la ressource (optionnel, si vous voulez cibler une ressource précise)
- **options** : des flags ou options supplémentaires (ex : -n pour le namespace, -o pour le format de sortie, etc.)

Quelques exemples

- *kubectl api-resources* : liste les services de l'API
- *kubectl explain <type de ressource>[.<sous-ressource>]* : expliquer un type de ressource (debug)
- *kubectl get pods -n mynamespace* : liste tous les pods de l'espace de nommage
- *kubectl get all -A* : liste toutes les ressources dans tous les espaces de nommage
- *kubectl apply -f fichier.yaml* : applique une configuration depuis un fichier

The slide features a light gray background with decorative elements in the corners. The top-left corner has a cluster of pink, purple, and orange circles. The top-right corner has blue, purple, and orange circles. The bottom-right corner has a cluster of pink, purple, and teal circles.

Rappels K8S - Partie 2/4

Les Principaux Contrôleurs

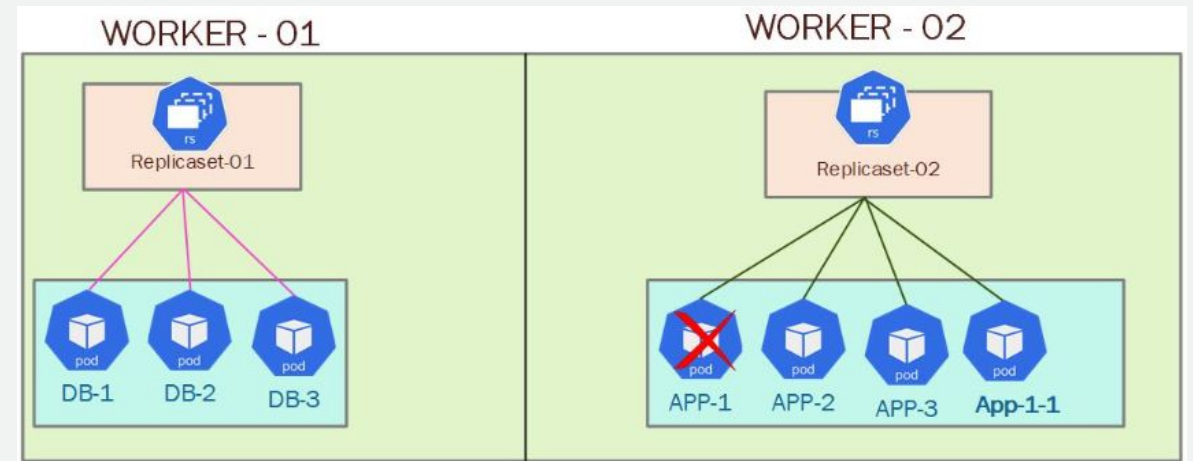
ReplicaSet Controller

Le *ReplicaSet Controller* est un composant du plan de contrôle de Kubernetes chargé de garantir qu'un nombre spécifié de Pods identiques (réplicas) sont toujours en cours d'exécution dans le cluster.

Le *ReplicaSet Controller* veille en permanence à ce que le nombre voulu de Pods soit toujours disponible dans le cluster, en créant ou supprimant des Pods selon les besoins, pour garantir la stabilité et la haute disponibilité de tes applications Kubernetes.

Le *ReplicaSet Controller* est essentiel pour assurer la tolérance aux pannes, la scalabilité et la gestion automatisée du cycle de vie des Pods dans Kubernetes.

Bien que l'on puisse créer un ReplicaSet seul, il est souvent utilisé indirectement via un Deployment, qui ajoute la gestion des mises à jour et des rollbacks.



Source: <https://shorturl.at/mB3ty>

Deployment Controller

Le Deployment Controller est un composant natif de Kubernetes chargé d'orchestrer le déploiement, la gestion, la mise à jour et la montée/descente en charge (scaling) des applications conteneurisées via l'objet Deployment.

Il s'appuie sur les ReplicaSets pour garantir la haute disponibilité et la résilience des Pods, tout en ajoutant des fonctionnalités avancées de gestion du cycle de vie applicatif.

Le Deployment Controller offre une gestion déclarative, fiable et automatisée du cycle de vie des applications, bien au-delà de ce que propose un ReplicaSet seul :

- **Sécurité et robustesse** : auto-réparation, tolérance aux pannes, zéro downtime lors des mises à jour.
- **Productivité** : simplifie la gestion des versions, des rollbacks et des montées en charge.
- **Traçabilité** : chaque changement est historisé et réversible.
- **Automatisation** : s'intègre parfaitement dans des pipelines CI/CD et des stratégies GitOps.

StatefulSet

Le contrôleur StatefulSet gère le déploiement et la gestion de pods stateful, c'est-à-dire des applications qui nécessitent :

- Une **identité réseau stable et unique** pour chaque pod (nom DNS, hostname)
- Un **stockage persistant et stable**, associé à chaque pod
- Un ordre précis lors du déploiement, de la mise à l'échelle, des mises à jour et de la suppression des pods

Contrairement à un Deployment (utilisé pour les applications stateless), le StatefulSet garantit que **chaque pod conserve son identité et son volume persistant**, même en cas de redémarrage ou de rescheduling sur un autre nœud

Cas d'usage typiques

- Bases de données distribuées : PostgreSQL, MySQL, MongoDB, Cassandra, etc.
- Systèmes de messagerie : Kafka, RabbitMQ
- Applications nécessitant un consensus ou une réplication, comme etcd ou ZooKeeper

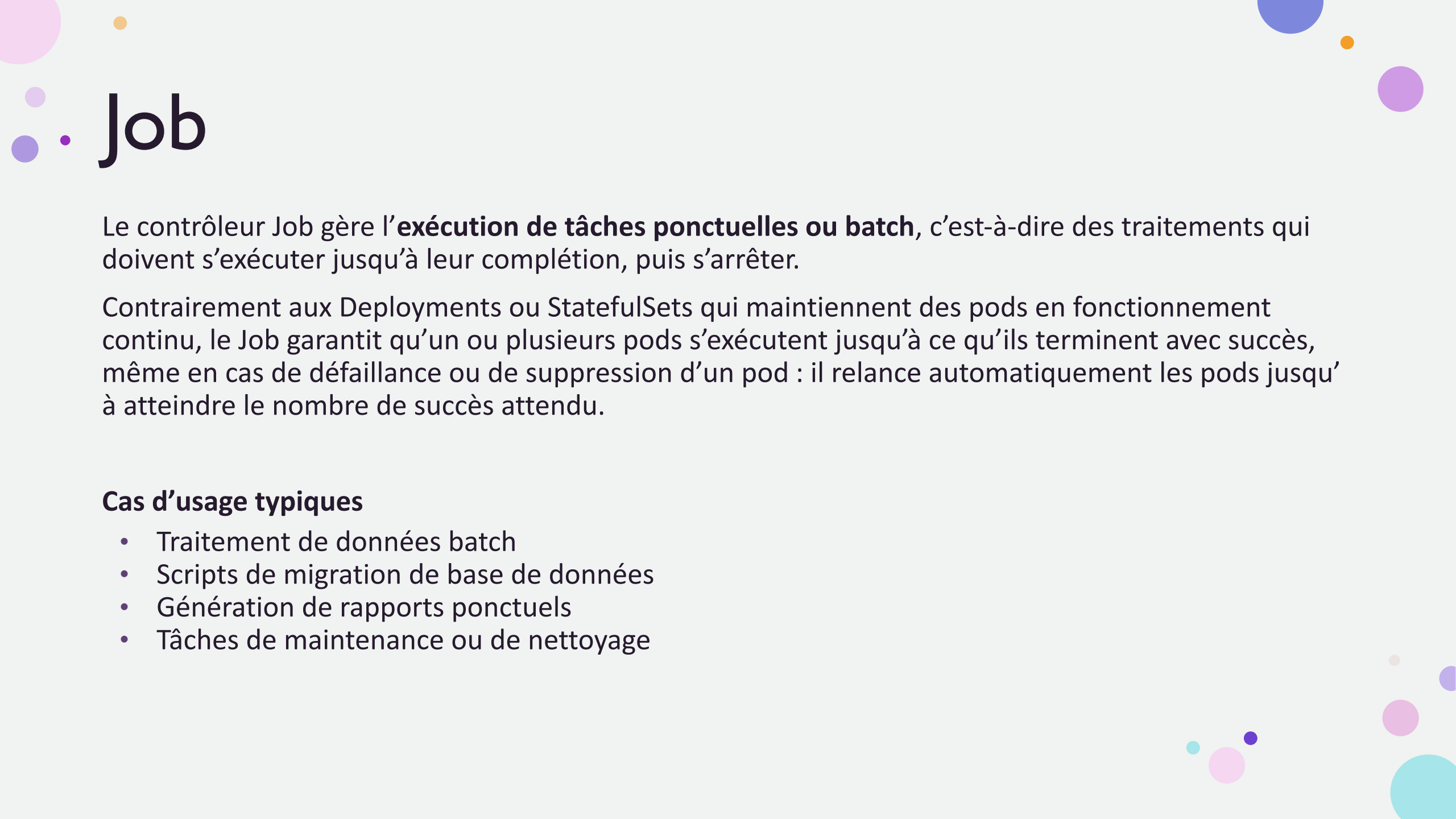
DaemonSet

Le contrôleur DaemonSet garantit qu'**un pod spécifique est déployé et fonctionne sur chaque nœud** d'un cluster Kubernetes, ou sur un sous-ensemble de nœuds si des sélecteurs sont utilisés. Ce comportement est essentiel pour les tâches qui doivent être présentes sur tous les nœuds, comme :

- La collecte de logs (ex : Fluentd, Filebeat)
- La surveillance (ex : Prometheus Node Exporter, Datadog)
- Les agents réseau (ex : CNI comme Calico, Cilium)
- Les outils de sécurité (ex : Falco, Sysdig)

Comportement

- Lorsqu'un DaemonSet est créé, Kubernetes déploie automatiquement un pod sur chaque nœud du cluster.
- Si un nouveau nœud rejoint le cluster, le DaemonSet y déploie automatiquement un pod correspondant.
- Si un nœud est supprimé, le pod associé est également supprimé.
- Les mises à jour peuvent être manuelles (OnDelete) ou progressives (RollingUpdate), selon la configuration.
- Il est possible de restreindre le déploiement à certains nœuds grâce à des labels et des nodeSelectors.



Job

Le contrôleur Job gère l'**exécution de tâches ponctuelles ou batch**, c'est-à-dire des traitements qui doivent s'exécuter jusqu'à leur complétion, puis s'arrêter.

Contrairement aux Deployments ou StatefulSets qui maintiennent des pods en fonctionnement continu, le Job garantit qu'un ou plusieurs pods s'exécutent jusqu'à ce qu'ils terminent avec succès, même en cas de défaillance ou de suppression d'un pod : il relance automatiquement les pods jusqu'à atteindre le nombre de succès attendu.

Cas d'usage typiques

- Traitement de données batch
- Scripts de migration de base de données
- Génération de rapports ponctuels
- Tâches de maintenance ou de nettoyage

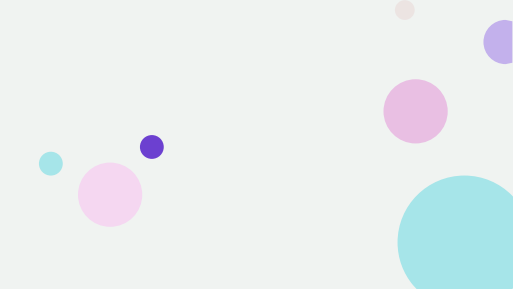


CronJob

Le contrôleur CronJob permet de planifier et d'exécuter des tâches récurrentes dans Kubernetes, à la manière de la commande cron sous Linux.

Il s'appuie sur le contrôleur Job pour lancer des pods selon un calendrier défini. Chaque exécution planifiée crée un Job indépendant, qui s'occupe d'exécuter la tâche jusqu'à son terme.

Cas d'usage typiques :

- Sauvegardes régulières de bases de données
 - Nettoyage périodique de ressources (logs, fichiers temporaires...)
 - Génération de rapports planifiés
 - Envoi automatique d'emails ou de notifications
- 

The slide features a light blue background with decorative elements consisting of various colored circles (pink, purple, blue, orange, teal) of different sizes scattered in the corners.

Rappels K8S - Partie 3/4

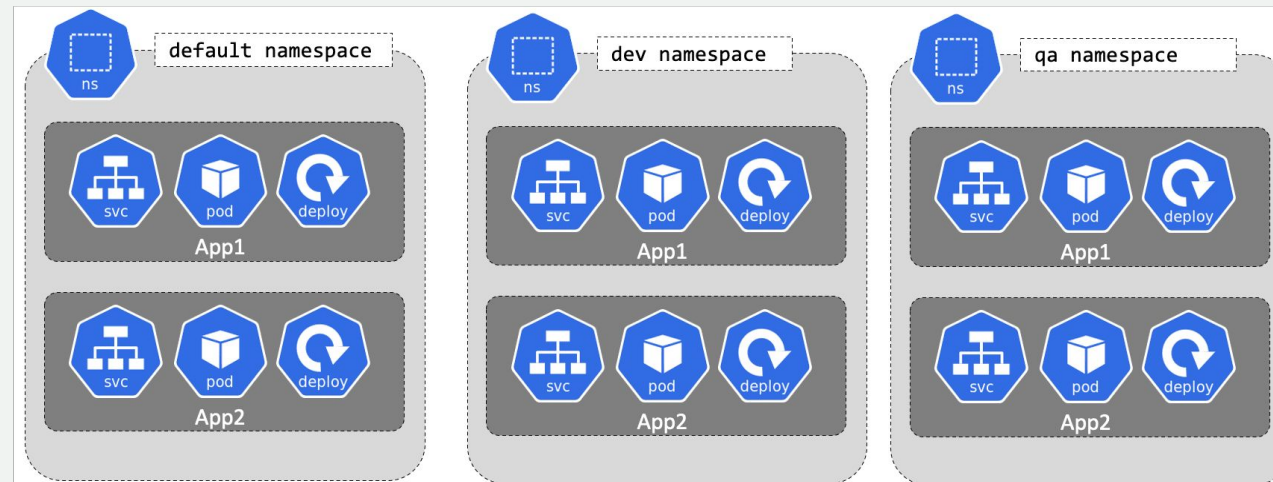
Organisation, Connectivité et Stockage

Namespaces

Un namespace est une **division logique** à l'intérieur d'un cluster Kubernetes : il permet de regrouper, isoler et organiser les ressources (Pods, Services, Deployments, etc.) dans des espaces distincts au sein du même cluster.

Chaque ressource qui « vit » dans un namespace possède un **nom unique** dans cet espace, mais ce nom peut être réutilisé dans d'autres namespaces.

Les namespaces sont essentiels pour **organiser, isoler, sécuriser et gérer** efficacement les ressources dans un cluster Kubernetes partagé ou complexe.



Source: <https://shorturl.at/No6zj>

Labels

Les labels (ou étiquettes) sont des paires clé/valeur attachées aux objets Kubernetes (Pods, Services, Deployments, Nodes, etc.).

Ils servent à identifier, organiser, regrouper et sélectionner des ressources selon des critères arbitraires définis par l'utilisateur ou les outils.

Syntaxe et règles

- **Clé/valeur :**
 - La clé et la valeur sont sensibles à la casse.
 - La clé peut avoir un préfixe (optionnel) de type DNS, séparé par /. Ex : app.kubernetes.io/name.
 - La clé ne doit pas dépasser 63 caractères (253 avec préfixe) et doit commencer/finir par un caractère alphanumérique.
 - Les caractères autorisés sont : lettres, chiffres, tirets (-), underscores (_), points (.).
- **Multiplicité :**
 - Un objet peut avoir plusieurs labels.
 - Un même label peut être partagé par plusieurs objets.

Services

Un Service est une ressource Kubernetes qui fournit une **abstraction réseau stable** pour accéder à un ou plusieurs Pods, indépendamment de leur cycle de vie.

Il agit comme un proxy ou un load balancer interne, permettant de communiquer avec des applications tournant dans des Pods sans se soucier de leurs adresses IP éphémères.

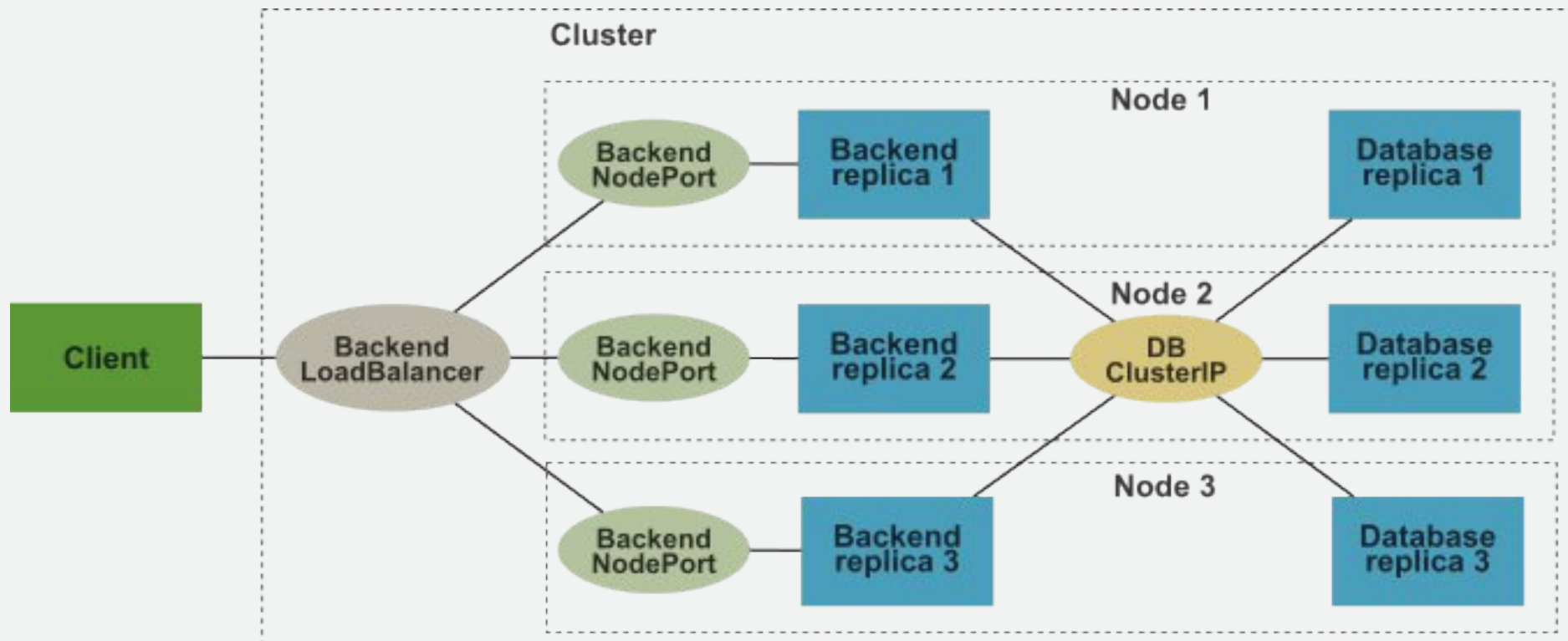
Principaux rôles

- **Adresse IP fixe et DNS stable:** Les Pods peuvent être créés, détruits ou déplacés : leur IP change. Un Service attribue une IP virtuelle et un nom DNS fixe, qui restent constants pour les clients.
- **Load balancing:** Un Service répartit automatiquement le trafic entre tous les Pods correspondants (ciblés par un label selector), assurant disponibilité et scalabilité.
- **Découverte et communication:** Les Services facilitent la découverte et la communication entre composants d'une application (frontend/backend, microservices, etc.) au sein du cluster.

Les Types de Service

Les services apportent des solutions aux problèmes de connectivités

4 types: ClusterIP, NodePort, LoadBalancer et External Name



API Gateway

Un point d'entrée unique pour tout le trafic, centralisant la gestion et la sécurité des accès.

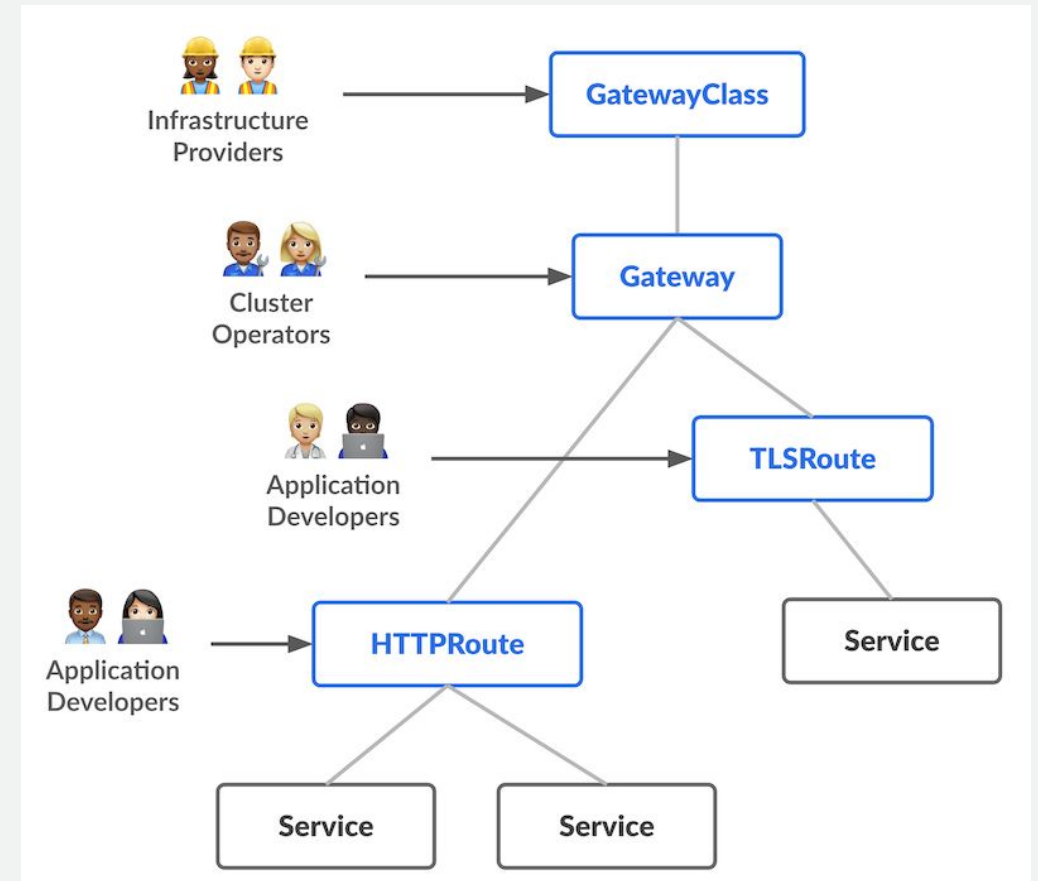
Des fonctionnalités avancées : authentification, autorisation, rate limiting, agrégation de réponses, transformation des requêtes/réponses, monitoring, etc.

Support multi-protocoles et routage applicatif avancé (split, canary, header-based, etc.).

Extensibilité : plugins, intégration avec des outils de sécurité, observabilité, etc.

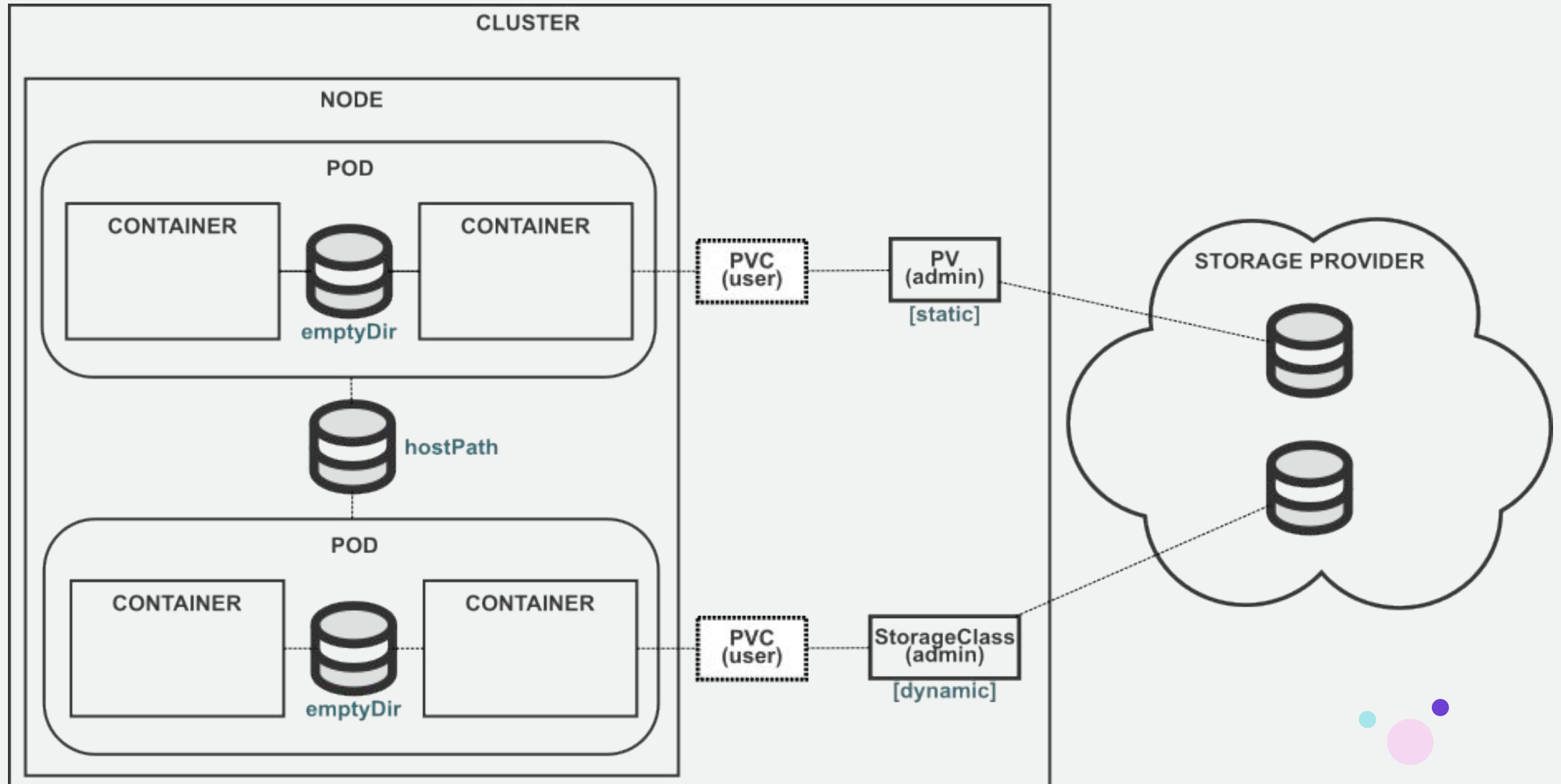
Séparation des rôles et RBAC : gestion des routes, des politiques et des accès par équipe ou par application.

Portabilité et standardisation : les nouveaux standards comme le Gateway API Kubernetes visent à uniformiser et enrichir ces usages pour tous les environnements (cloud, on-prem, hybride).



Source: <https://kubernetes.io/>

Volumes - PVC, PV, StorageClass



Persistent Volume (PV)

Un PersistentVolume (PV) est une ressource de stockage abstraite et indépendante dans un cluster Kubernetes.

Il permet de fournir un espace de stockage persistant aux applications, c'est-à-dire un stockage qui subsiste même si les Pods qui l'utilisent sont arrêtés, supprimés ou recréés.

Objectifs et utilité

- **Abstraction du stockage** : Le PV masque les détails de la technologie de stockage sous-jacente (NFS, iSCSI, disque cloud, stockage local, etc.) pour les utilisateurs et les applications. Cela facilite la portabilité et la gestion du stockage dans le cluster.
- **Persistance des données** : Contrairement aux volumes classiques liés au cycle de vie d'un Pod, un PV existe indépendamment et conserve les données au-delà de la durée de vie des Pods.
- **Gestion centralisée** : Les administrateurs peuvent provisionner des PVs de façon statique (manuelle) ou dynamique (automatique via des StorageClass), permettant une gestion flexible et évolutive du stockage dans le cluster.

Persistent Volume Claim (PVC)

Un PersistentVolumeClaim (PVC) est une ressource Kubernetes qui représente une demande de stockage persistante, formulée par un utilisateur ou une application.

Le PVC permet de spécifier la quantité de stockage souhaitée, le mode d'accès (lecture/écriture, lecture seule, etc.) et éventuellement la classe de stockage désirée, sans se soucier des détails techniques du volume sous-jacent.

Utilité

- Le PVC permet aux utilisateurs et aux Pods de consommer du stockage persistant de façon déclarative, sans connaître la technologie de stockage utilisée (NFS, disque cloud, etc.).
- Il facilite la portabilité et l'automatisation : les développeurs décrivent seulement leurs besoins, et Kubernetes s'occupe de la gestion du stockage.

Storage Class

Une Storage Class dans Kubernetes est une ressource qui permet aux administrateurs de définir différentes "classes" de stockage, chacune correspondant à un type, une qualité de service, des performances ou des politiques spécifiques de stockage.

Elle sert de modèle ou de profil pour le **provisionnement dynamique des volumes persistants (PV)** en fonction des besoins des applications.

Utilité

- **Provisionnement dynamique** : Grâce aux StorageClasses, les volumes persistants (PV) peuvent être créés automatiquement dès qu'un PersistentVolumeClaim (PVC) en fait la demande, sans intervention manuelle.
- **Abstraction et flexibilité** : Les utilisateurs peuvent choisir la classe de stockage la plus adaptée à leur workload (par exemple : SSD pour la performance, HDD pour l'archivage, NFS pour le partage, etc.) simplement en spécifiant le nom de la StorageClass dans leur PVC.
- **Gestion centralisée** : Les administrateurs définissent les types de stockage disponibles et leurs paramètres (provisionneur, options, politiques de rétention, etc.), ce qui simplifie la gestion et le respect des standards de l'entreprise.

The slide features a light gray background with decorative elements in the corners. The top-left corner contains a large pink circle, a small orange circle, a small light purple circle, and a small dark purple circle. The top-right corner contains a large blue circle, a small orange circle, and a medium purple circle. The bottom-right corner contains a small teal circle, a medium pink circle, a small dark purple circle, a medium pink circle, and a large teal circle.

Rappel K8S Partie 4/4

Debug

Problèmes courants

Problèmes de ressources

- Sur-utilisation ou sous-utilisation du CPU/mémoire : provoque des pods en attente ou des ralentissements.
- CrashLoopBackOff : redémarrages en boucle des pods, souvent liés à des erreurs de configuration, de ressources insuffisantes ou de dépendances manquantes.
- ErrImagePull : échec d'extraction d'une image de conteneur, souvent à cause d'un nom d'image incorrect ou d'un problème d'accès au registre.

Problèmes de connectivité réseau

- Pods incapables de communiquer entre eux ou avec l'extérieur (ex. : port VXLAN en conflit, politiques réseau trop restrictives).
- Services inaccessibles malgré des pods fonctionnels (erreurs de mapping de ports, problèmes d'Ingress, DNS interne mal configuré).

Problèmes courants

Problèmes de plan de contrôle

- Nœuds indisponibles ou défaillants.
- Services du plan de contrôle (API server, etcd, scheduler) en panne ou instables.
- Problèmes de certificats ou de configuration empêchant la communication avec l'API server.

Problèmes de déploiement

- Pods bloqués à l'état « ContainerCreating » (souvent lié à l'image de pause ou à la configuration réseau).
- Déploiements échoués après une mise à jour (rollback nécessaire, erreurs dans les logs, sondes de disponibilité non satisfaites).

Méthodes

Vérifier l'état global du cluster

- *kubectl cluster-info* : informations sur le plan de contrôle.
- *kubectl get componentstatus* : santé des composants principaux (API server, etcd, scheduler).

Inspecter les nœuds

- *kubectl get nodes -o wide*
- *kubectl describe node <node-name>* : détails sur la santé, les ressources, les taints, etc.

Analyser les pods

- *kubectl get pods -A -o wide --field-selector status.phase!=Running* : liste des pods non sains.
- *kubectl describe pod <pod-name>* : événements récents, raisons d'échec, détails sur les conteneurs.
- *kubectl logs <pod-name>* : logs applicatifs (stdout/stderr).
- *kubectl exec -it <pod-name> -- /bin/sh* : shell interactif pour diagnostiquer de l'intérieur.
- *kubectl port-forward pod/mon-app-7d8f 8888:8080*

Méthodes

Examiner les services et la connectivité

- *kubectl get services, kubectl describe service <service-name>*
- *kubectl get endpoints my-service -o yaml*
- *kubectl describe ingress <ingress-name>*
- Vérification des network policies : *kubectl get networkpolicies, kubectl describe networkpolicy <policy-name>*.

Surveillance et événements

- *kubectl get events --sort-by=.metadata.creationTimestamp -n <namespace>* : chronologie des événements récents.
- *kubectl top pods, kubectl top nodes* : consommation de ressources.

Lab “Prise en main”

Pour illustrer les capacités de K8S, Kustomize et Helm, nous allons nous appuyer sur une application “fil rouge”. Il s’agit d’une application permettant de publier des news, constituée des briques suivantes:

- **Frontend** — Interface statique servie par Nginx, avec reverse proxy vers le backend
- **Backend** — API REST en Flask (Python) pour gérer des articles
- **Redis** — Base de données clé-valeur pour le stockage des articles
- **Cleaner** — Script shell de maintenance qui supprime les articles expirés

Il vous est demandé de suivre le lab 1 du dossier k8s avec pour objectif de créer à partir du fichier docker-compose.yml, les fichiers de ressources k8s permettant un déploiement sur le cluster GKE.

Ce travail préparera les labs de customisation (Kustomize) et de packaging (Helm) pour la suite.

The slide features a light gray background with decorative elements in the corners. The top-left corner contains a cluster of circles in shades of pink, purple, and orange. The top-right corner has circles in blue, purple, and orange. The bottom-right corner is decorated with circles in teal, pink, and purple. The text is centered in a dark purple, sans-serif font.

Rappels sur Kustomize

Présentation Rapide

Kustomize est un outil natif de **gestion de configuration** pour Kubernetes, intégré directement à kubectl depuis la version 1.14.

Il permet de personnaliser et de gérer les fichiers YAML de ressources Kubernetes **sans recourir à des templates ou à la duplication de fichiers**, contrairement à des outils comme Helm qui utilisent un système de templates avec des valeurs dynamiques.

L'objectif principal de Kustomize est d'appliquer le **principe DRY** (Don't Repeat Yourself) aux manifests Kubernetes.

- Plutôt que de dupliquer des fichiers YAML pour chaque environnement (développement, staging, production), Kustomize propose de définir une base commune et d'y appliquer des modifications ciblées via des overlays ou des patches.
- Cela simplifie la maintenance et l'évolution des configurations, tout en gardant les fichiers d'origine intacts et réutilisables.

A cluster of circles in shades of pink, purple, and orange in the top-left corner.

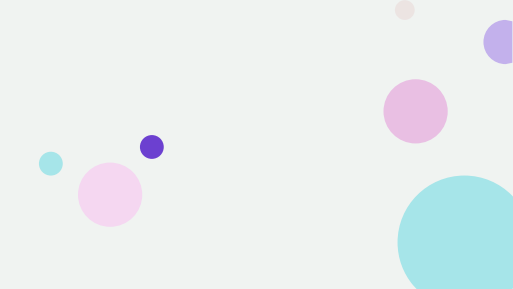
Avantages

Natif et simple : Utilisable directement avec kubectl, sans installation supplémentaire depuis la v1.14.

Pas de templating : Utilise une approche déclarative et sans logique de template, ce qui facilite la validation et la compréhension des manifests.

Gestion multi-environnements : Permet de gérer facilement plusieurs variantes de configuration pour différents environnements, équipes ou clients.

Sécurité : Facilite la gestion des secrets et des ConfigMaps sans les inclure directement dans les manifests versionnés.

A cluster of circles in shades of pink, purple, and teal in the bottom-right corner.

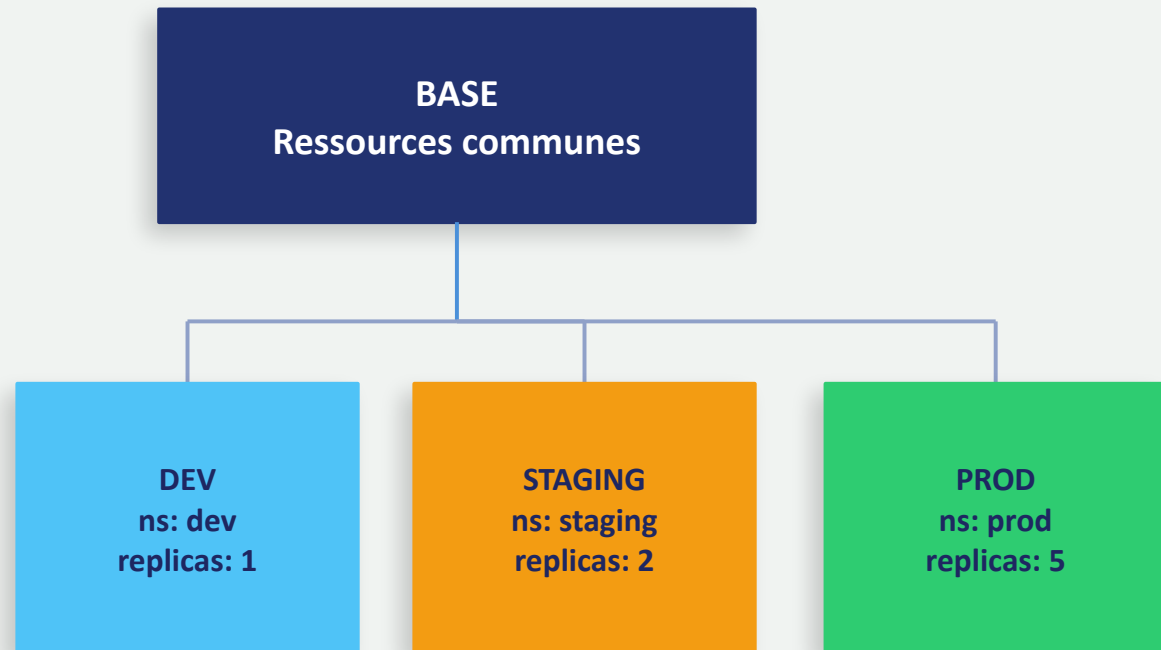
Le problème de la réutilisation

Objectifs:

- Base commune unique
- Overlays par environnement
- Zéro duplication
- Traçabilité Git

```
app/  
├── base/  
│   ├── kustomization.yaml  
│   ├── deployment.yaml  
│   └── service.yaml  
├── overlays/  
│   ├── dev/  
│   │   └── kustomization.yaml  
│   ├── staging/  
│   │   └── kustomization.yaml  
│   └── prod/  
│       ├── kustomization.yaml  
│       └── patch-replicas.yaml
```

```
# deployment-dev.yaml  
replicas: 1  
image: nginx:1.24  
  
# deployment-prod.yaml ← copié/collé !  
replicas: 5  
image: nginx:1.25
```



L'overlay référence la base et n'applique que les différences.

Fonctionnement

Fichier kustomization.yaml : Au cœur de Kustomize, ce fichier décrit comment assembler et personnaliser les ressources Kubernetes. Il référence les ressources de base, les patches à appliquer, les générateurs de ConfigMaps et de Secrets, ainsi que les labels ou annotations à ajouter.

Bases et overlays - La structure d'un projet Kustomize repose sur une hiérarchie :

- **Base** : Ensemble de ressources communes à tous les environnements.
- **Overlay** : Personnalisation spécifique à un environnement, qui s'appuie sur la base (ex : changer le nombre de réplicas, ajouter un label, modifier une image Docker).

Transformateurs : Appliquent des modifications globales sur les ressources (ajout de préfixes/suffixes, labels, annotations, etc.).

Patches : Modifient de façon ciblée des champs spécifiques dans les ressources YAML existantes, sans dupliquer l'intégralité du fichier.

Générateurs : Permettent de créer des ConfigMaps et des Secrets à partir de fichiers ou de valeurs littérales, sans avoir à écrire les ressources à la main.

Transformers communs

Transformer	Rôle principal	Exemple d'utilisation
commonLabels	Ajouter/modifier des labels	app: my-app, env: prod
commonAnnotations	Ajouter/modifier des annotations	version: "1.0.0", branch: master
namespace	Définir le namespace commun	staging, production
namePrefix	Ajouter un préfixe aux noms de ressources	dev-, test-
nameSuffix	Ajouter un suffixe aux noms de ressources	-v1, -prod
images	Modifier nom/tag/digest d'images Docker	nginx:1.25, my-app:2.0

Cohérence : Toutes les ressources reçoivent les mêmes labels, annotations, etc.

Centralisation : Les modifications se font à un seul endroit (kustomization.yaml).

Gain de temps : Plus besoin de modifier chaque fichier YAML individuellement.

Évolutivité : Idéal pour gérer plusieurs environnements ou équipes

Patches

Kustomize propose un système de patches permettant de modifier les fichiers de ressources Kubernetes sans toucher aux fichiers de base.

- Cette approche favorise **la réutilisation, la maintenabilité et la séparation des préoccupations** entre la configuration commune et les personnalisations spécifiques à un environnement ou à un besoin particulier.

Opérations principales :

- **Ajout ou modification de champs** : Vous pouvez ajouter de nouveaux champs ou modifier la valeur de champs existants en ne spécifiant que les champs concernés dans le patch.
- **Fusion intelligente** : Pour certains objets (comme les listes de conteneurs dans un Deployment), la fusion se fait en fonction de la clé (par exemple, le nom du conteneur), ce qui permet d'ajouter ou de modifier des éléments sans écraser l'ensemble de la liste.
- **Suppression d'éléments** : Possible via la directive spéciale `$patch: delete` dans le patch YAML, mais ce n'est pas aussi flexible que le JSON Patch pour la suppression de champs ou d'éléments de tableau.

Limitations :

- Ne permet pas de manipuler précisément les tableaux (par exemple, supprimer un élément à un index donné).
- Ne permet pas d'opérations conditionnelles ou de tests.

Transfo / Patch : Examples

```
apiVersion:
kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - deployment.yaml
  - service.yaml

commonLabels:
  app: my-app

images:
  - name: nginx
    newTag: "1.24"
```

base/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

bases:
  - ../../base

namespace: production

namePrefix: prod-

patches:
  - path: patch-replicas.yaml
    target:
      kind: Deployment
```

overlays/prod/kustomization.yaml

Générateurs

Un générateur Kustomize est un mécanisme qui crée automatiquement des ressources Kubernetes, principalement des ConfigMaps et des Secrets, sans avoir à écrire leur manifest YAML à la main.

Il suffit de fournir des sources (fichiers, variables littérales, variables d'environnement) et il produit la ressource correspondante au moment du build.

L'avantage clé est le hash suffix automatique : chaque fois que le contenu change, le nom change aussi (webapp-config-7d4f9b2m → webapp-config-9k2p1x4n), ce qui force Kubernetes à redéployer les pods qui consomment cette ressource — garantissant qu'ils n'utilisent jamais une config obsolète.

Générateur : Exemples

```
configMapGenerator:  
- name: app-config  
  files:  
    - config/app.properties  
  
- name: env-config  
  literals:  
    - LOG_LEVEL=info  
    - MAX_RETRIES=3  
  
options:  
  disableNameSuffixHash: true
```

configMapGenerator

```
secretGenerator:  
- name: db-credentials  
  type: Opaque  
  literals:  
    - username=admin  
    - password=$(DB_PASS)
```

secretGenerator

Components

Les components (composants) sont une fonctionnalité avancée de Kustomize permettant **de factoriser et de réutiliser des morceaux de configuration** dans différents overlays, sans dupliquer le code.

Ils sont conçus pour **modulariser les personnalisations**, notamment lorsque certaines fonctionnalités (monitoring, debug, dashboards, etc.) doivent être activées ou désactivées selon l'environnement ou le besoin.

- Un component est un répertoire avec un kustomization.yaml spécial, dont le type est kind: Component (et non Kustomization).
- Il peut contenir des ressources, des patches, des générateurs, des transformers, etc., comme un overlay classique.
- Contrairement à un overlay, un component n'est pas destiné à être appliqué seul, mais à être inclus dans d'autres overlays via le champ components.
- Un component s'applique après l'accumulation des ressources de l'overlay parent, ce qui permet de transformer ou d'étendre les ressources déjà présentes, et de composer plusieurs components séquentiellement.

Components - Avantages

Réutilisabilité : Centraliser des fonctionnalités optionnelles (debug, monitoring, RBAC, dashboards, etc.) pour les activer facilement dans un ou plusieurs overlays sans duplication.

Modularité : Construire des overlays complexes à partir de briques élémentaires, activables à la demande.

Simplicité : Réduire la complexité et la maintenance des overlays, en évitant la répétition de patches ou de ressources partagées.

Components - Examples

```
apiVersion:
kustomize.config.k8s.io/v1alpha1
kind: Component
resources:
  - servicemonitor.yaml
patches:
  - path: patch-annotations.yaml
```

components/monitoring/kustomization.yaml

```
bases: [../../base]
components:
  - ../../components/monitoring
  - ../../components/rbac
```

overlays/prod/kustomization.yaml

Exemples

Cf. Contenu du **dossier kustomize**

Preview : *kubectl kustomize overlays/development*

Application : *kubectl apply -k overlays/development*

Suppression : *kubectl delete -k overlays/development*

Validation : *kubectl kustomize overlays/production | kubectl apply --dry-run=client -f -*

Cheat Sheet

Build

```
kubectl kustomize overlays/<env>
```

Apply

```
kubectl apply -k overlays/<env>
```

Diff

```
kubectl diff -k overlays/<env>
```

Patch merge

```
patches: [{path: patch.yaml, target: {kind: Deployment}}]
```

Patch JSON6902

```
patches: [{path: p.yaml, target: {group: apps, version: v1, kind: Deployment}}]
```

Image update

```
images: [{name: nginx, newTag: "1.25"}]
```

Namespace

```
namespace: production
```

Prefix/Suffix

```
namePrefix: prod-    nameSuffix: -v2
```

Component

```
components: [../../components/monitoring]
```

Labels

```
commonLabels: {app: my-app, env: prod}
```

Lab Kustomize

Dans le premier Lab, vous avez créé des fichiers YAML "en dur". Si demain, vous devez déployer en Production avec 10 répliques et en Dev avec 1 seul, vous devriez copier-coller et modifier tous les fichiers... C'est ingérable.

Kustomize (intégré nativement à kubectl) résout ce problème par un système d'héritage (Base & Overlays).

Il vous est demandé ici de suivre le lab2 du dossier k8s afin de “kustomizer” le déploiement.

The slide features a light gray background with decorative elements in the corners. The top-left corner contains a cluster of circles in shades of pink, purple, and orange. The top-right corner has circles in blue, purple, and orange. The bottom-right corner is decorated with circles in teal, pink, and purple. The text is centered in a dark purple font.

Helm

Présentation Générale

Vue d'Ensemble

Helm est à Kubernetes ce que apt est à Debian

Il offre une capacité de versionnement pour une application basée sur des micro-services

Il facilite le déploiement

- Il évite notamment les nombreux `$kubectl apply -f ressource.yaml`
- Il centralise les versions des micro-services

```
$ curl -fsSL -o get_helm.sh  
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-4  
$ chmod 700 get_helm.sh  
$ ./get_helm.sh
```

Artifact Hub est pour Kubernetes ce que Docker Hub est pour docker

- <https://artifacthub.io/>

Helm tire toutes les dépendances requises lors de l'installation d'un chart

Demo: installation de Wordpress par Helm

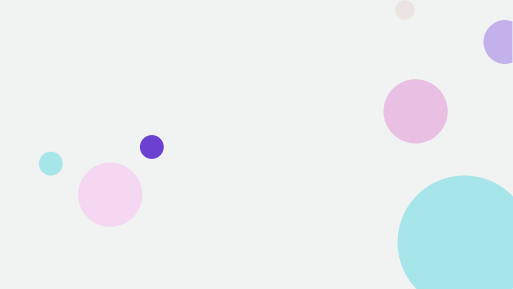
- Ajout du repo
- Installation



Introduction

Helm est le gestionnaire de paquets officiel pour Kubernetes, conçu pour simplifier et automatiser le déploiement, la gestion et la mise à jour d'applications dans des environnements Kubernetes.

Il joue un rôle similaire à celui d'outils comme apt ou yum dans le monde des distributions Linux, mais appliqué à l'écosystème Kubernetes, en permettant de gérer des applications complexes sous forme de paquets appelés charts.



Motivations

Déployer une application sur Kubernetes nécessite généralement de manipuler de nombreux fichiers YAML, souvent redondants et difficiles à maintenir à grande échelle.

Helm répond à cette problématique en proposant :

- Un système de packaging (chart) qui regroupe tous les fichiers nécessaires au déploiement d'une application, ainsi que ses dépendances et sa configuration.
- Un moteur de templating qui permet de paramétrer dynamiquement les ressources Kubernetes, évitant la duplication des fichiers et facilitant la gestion multi-environnements.
- La gestion du cycle de vie des applications : installation, mise à jour, rollback, suppression, le tout de façon versionnée et traçable.

Historique

2015 - Naissance (v1) : Créé par l'équipe Deis (Hackathon) pour résoudre la complexité des fichiers YAML Kubernetes.

2016 - Maturité (v2) : Introduction de Tiller (composant serveur). Helm rejoint la CNCF. L'outil devient le standard, mais Tiller introduit des défis de sécurité (droits RBAC trop larges).

2019 - Modernisation (v3) : Suppression de Tiller. Helm utilise désormais l'architecture "client-only" et s'appuie nativement sur la sécurité de l'API Kubernetes.

Nov 2025 - Évolution vers le GitOps (v4) : Meilleure intégration avec ArgoCD/Flux, observabilité et sécurité améliorées, meilleures performances.

Aujourd'hui : Projet "Graduated" de la CNCF. Helm est la brique indispensable pour le GitOps et la gestion de packages Cloud Native.

Helm v4 – Évolutions majeures (I / 2)

Server-Side Apply (SSA) par défaut :

- Remplace l'ancien algorithme de "Three-way merge".
- Kubernetes gère désormais lui-même les conflits de champs entre Helm et d'autres contrôleurs (ex: GitOps, Autoscalers).
- Bénéfice : Déploiements plus fiables et meilleure cohabitation avec ArgoCD/Flux.

Système de Plugins Refondu (WebAssembly) :

- Introduction du support WASM pour les plugins.
- Exécution sécurisée dans un bac à sable (sandbox) et portabilité totale.
- Les "Post-renderers" (utilisés pour Kustomize) deviennent des plugins à part entière.

Helm v4 – Évolutions majeures (2 / 2)

Sécurité & Supply Chain :

- Support natif de l'installation par Digest OCI (sha256) au lieu de simples tags.
- Garantit l'immuabilité absolue de ce qui est déployé en production.

Observabilité avec kstatus :

- Intégration d'un nouveau moteur de suivi (watcher) pour la commande --wait.
- Détection intelligente des états complexes (Progressing, Degraded) même pour les CRDs.

Changements de rupture :

- Renommage des flags pour plus de clarté : --atomic devient --rollback-on-failure.
- Suppression définitive des anciens mécanismes de migration Helm v2.



Avantages

Déploiement rapide et standardisé d'applications complexes.

Gestion centralisée des configurations et des dépendances.

Possibilité de **versionner**, mettre à jour et restaurer des applications facilement.

Forte adoption communautaire et nombreux charts disponibles en open source.





Cas d'usage typiques

Déploiement d'applications multi-tiers (bases de données, backends, frontends) en une seule commande.

Automatisation des déploiements CI/CD dans des environnements Kubernetes.

Standardisation et partage de bonnes pratiques de déploiement via des charts réutilisables.



Concepts Essentiels

Chart : Paquet contenant toutes les ressources Kubernetes nécessaires à une application (déploiements, services, configurations, etc.).

Release : Instance d'un chart déployée sur un cluster Kubernetes, associée à une configuration spécifique.

- Les releases peuvent avoir différentes versions

Repository : Emplacement (distant ou local) où sont stockés et partagés les charts Helm.

Values : Fichier de valeurs permettant de personnaliser la configuration d'un chart lors de son déploiement.

Installation - Linux

Par un script (recommandé)

- `curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-4`
- `chmod +x get_helm.sh`
- `./get_helm.sh`

Par le gestionnaire de paquets

- `sudo apt-get install helm`
- `sudo yum install helm`
- `sudo snap install helm --classic`

Installation – Windows & Mac

Par script (recommandé) – Comme sur linux

Windows: Par chocolatey

- *choco install kubernetes-helm*

Mac: Par Homebrew

- *brew install helm*

Commandes Essentielles

Gestion des Charts

helm create <name> : Crée une structure de Chart

helm package <chart-path> : Empaquete dans un fichier d'archive versionné.

helm lint <chart> : Effectue des tests pour examiner un Chart et identifie les problèmes potentiels

helm show all <chart> : Inspecte le contenu d'un Chart

helm show values <chart> : Affiche le contenu du fichier values.yaml

helm pull <chart> : Télécharger un Chart

helm pull <chart> --untar=true : Télécharger et désarchiver un Chart

helm pull <chart> --verify : Télécharger en vérifiant préalablement un Chart

helm pull <chart> --version <number> : Télécharger une version spécifique d'un Chart

helm dependency list <chart> : Afficher les dépendances d'un Chart

Commandes Essentielles

Installation / Désinstallation d'Applications

helm install <name> <chart> : Installation d'un Chart avec un nom (Release)

helm install <name> <chart> --namespace <namespace> : Installation d'un Chart dans un NS donné

helm install <name> <chart> --set key1=val1,key2=val2 : Installation d'un Chart avec des valeurs spécifiques

helm install <name> <chart> --values <yaml-file/url> : Installation d'un Chart avec un fichier de valeurs

helm install <name> <chart> --dry-run --debug : Installation dry-run d'un Chart

helm install <name> <chart> --verify : Vérification d'un Chart avant son utilisation

helm install <name> <chart> --dependency-update : Mise à jour des dépendances avant installation

helm uninstall <name> : Désinstallation d'un Chart

helm uninstall <release-name> --namespace <namespace> : Désinstallation d'un Chart d'un NS donné

Commandes Essentielles

Mises à jour et Rollbacks

helm upgrade <release> <chart> : Met à jour une release

helm upgrade <release> <chart> --rollback-on-failure : Met à jour une release avec Rollback si besoin

helm upgrade <release> <chart> --dependency-update : Met à jour une release avec MàJ des dépendances

helm upgrade <release> <chart> --version <version_number> : Met à jour une release dans une version donnée

helm upgrade <release> <chart> --values <yaml file/url> : Met à jour une release avec un fichier de valeurs

helm upgrade <release> <chart> --set key1=val1,key2=val2 : Met à jour une release avec des valeurs

helm upgrade <release> <chart> --force : Force la MàJ d'une release

helm rollback <release> <revision> : Rollback vers une version spécifique

helm rollback <release> <revision> --cleanup-on-fail : Rollback avec nettoyage en cas de problème

Commandes Essentielles

Gestion des Repos

helm repo add <repo-name> <url> : Ajouter un repo

helm repo list : Lister les repos

helm repo update : Mettre à jour les informations concernant les repos

helm repo remove <repo_name> : Supprimer un repo

helm repo index <DIR> : Génération d'un index à partir d'un dossier

helm repo index <DIR> --merge : Fusionner l'index généré avec un index existant

helm search repo <keyword> : Recherche d'un repo à partir d'un mot de clef

helm search hub <keyword> : Recherche d'un repo à partir d'un mot de clef sur Artifact Hub

Commandes Essentielles

Monitoring des Releases (1 / 2)

helm list : Liste les release dans le NS par défaut

helm list --all (-a): Liste toutes les releases sans filtres

helm list --all-namespaces (-A) : Liste toutes les releases de tous les NS

helm list -l key1=value1,key2=value2 : Liste toutes les releases en utilisant des filtres

helm list --date : Liste toutes les releases en triant par date

helm list --deployed : Liste toutes les releases déployées

helm list --pending : Liste toutes les releases en attente

helm list --failed : Liste toutes les releases défaillantes

Commandes Essentielles

Monitoring des Releases (2 / 2)

helm list --uninstalled : Liste toutes les releases (Si l'option ***--keep-history*** a été utilisé)

helm list --superseded : Liste toutes les releases surchargées

helm list -o yaml : Liste toutes les releases dans différents formats (**table**, json, yaml)

helm status <release> : Affiche les statut d'une release

helm status <release> --revision <number> : Affiche les statut d'une release d'une révision spécifique

helm history <release> : Affiche l'historique d'une release

helm env : Affiche les variables d'environnement utilisées par Helm

Commandes Essentielles

Informations sur les Releases

helm get all <release> : Affiche toutes les informations détaillées d'une release

helm get hooks <release> : Télécharge les hooks au format yaml pour une release donnée

helm get manifest <release> : Télécharge le manifest au format yaml pour une release donnée

helm get notes <release> : Télécharge les notes au format yaml pour une release donnée.

helm get values <release> : Télécharge les valeurs au format yaml pour une release donnée.



Commandes Essentielles

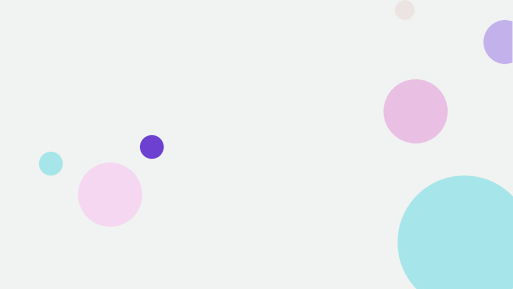
Gestion des Plugins

helm plugin install <path/url> : Installe un Plugin

helm plugin list : Liste les plugins

helm plugin update <plugin> : Met à jour un Plugin

helm plugin uninstall <plugin> : Désinstalle un Plugin



Templates

Helm utilise le moteur de templates de Go (Go templating engine) enrichi par la bibliothèque de fonctions Sprig, ce qui permet de générer dynamiquement des fichiers manifestes Kubernetes à partir de modèles paramétrables.

Syntaxe Go Template : Les fichiers dans le dossier templates/ d'un chart Helm sont principalement des fichiers YAML dans lesquels on insère des expressions entre doubles accolades `{{ ... }}`.

Variables et objets intégrés : Helm expose plusieurs objets spéciaux dans le contexte du template, comme `.Values` (valeurs utilisateur), `.Release` (informations sur la release), `.Chart` (métadonnées du chart), etc.

Fonctions Sprig : Helm intègre la librairie Sprig, offrant de nombreuses fonctions utilitaires pour manipuler des chaînes, des listes, des dates, etc. Exemples :

- `{{ upper .Values.env }}` pour mettre en majuscule une variable.
- `{{ default "valeurParDefaut" .Values.option }}` pour définir une valeur par défaut

Templates

Contrôle de flux : On peut utiliser des structures de contrôle comme les conditions (**if**, **else**) et les boucles (**range**) pour générer dynamiquement des parties du YAML.

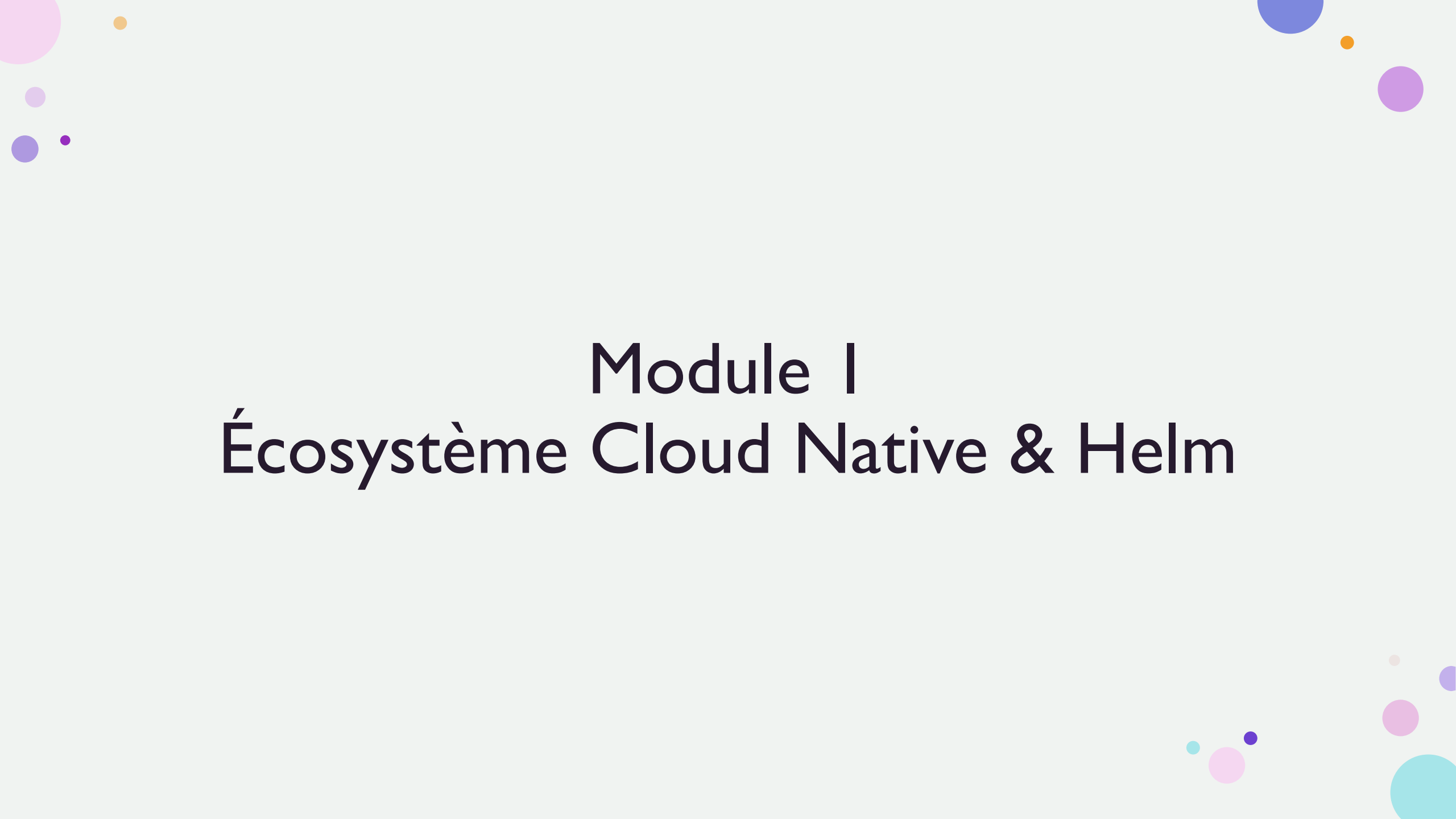
Templates nommés et helpers : Il est possible de factoriser du code réutilisable dans des fichiers comme `_helpers.tpl` grâce à la directive `define/template`.

```
{{- if .Values.autoscaling.enabled }}  
# YAML block for autoscaling  
{{- end }}
```

```
{{- range .Values.ports }}  
- containerPort: {{ . }}  
{{- end }}
```

```
{{- define "monchart.fullname" -}}  
{{ printf "%s-%s" .Release.Name .Chart.Name }}  
{{- end }}
```

<https://masterminds.github.io/sprig/>

The slide features a light gray background with decorative elements in the corners. The top-left corner contains several circles in shades of pink, purple, and orange. The top-right corner has circles in blue, orange, and purple. The bottom-right corner is decorated with circles in teal, pink, and purple. The main text is centered on the slide.

Module I

Écosystème Cloud Native & Helm

Le problème du "YAML pur"

La duplication de code (Copier-Coller) : Pour déployer la même application en Dev, Staging et Prod, on duplique souvent 90% du YAML.

La rigidité : Pas de variables, pas de logique conditionnelle native dans les fichiers manifestes K8s.

La gestion des versions : Comment savoir quelle version de l'application et de sa configuration est actuellement en ligne ?

L'atomicité : Kubernetes déploie des ressources individuellement. Si un Service passe mais que le Deployment échoue, l'état du cluster est incohérent.

Qu'est-ce que Helm ?

Définition : Le gestionnaire de paquets (Package Manager) pour Kubernetes.

Équivalence : Helm est à Kubernetes ce que apt est à Debian ou npm à Node.js.

Le concept de "Chart" : Un ensemble de fichiers YAML packagés et versionnés.

Le concept de "Release" : Une instance d'un Chart exécutée dans un cluster avec une configuration spécifique.

Trois piliers :

- **Créer** des packages réutilisables.
- **Partager** via des dépôts (Repositories/Registries).
- **Gérer** le cycle de vie (Installation, Mise à jour, Rollback).

Architecture Helm v4 (1 / 2)

L'héritage de la v3 (Tillerless) :

- Toujours aucun composant serveur permanent.
- Utilisation native du RBAC de l'utilisateur via le kubeconfig.

Moteur de déploiement : Server-Side Apply (SSA) :

- v3 : Helm calculait les différences en local (Three-way merge) et envoyait tout le YAML.
- v4 : Helm envoie les intentions à l'API Kubernetes qui gère elle-même la fusion des ressources.
- Bénéfice : Meilleure gestion des conflits avec les contrôleurs externes (ArgoCD, HPA).

Architecture Helm v4 (2 / 2)

Stockage des Releases :

- L'état est toujours stocké dans des Secrets (ou ConfigMaps) au sein du cluster.
- Nouveauté v4 : Compression optimisée et gestion des versions de releases plus légère pour les clusters à haute densité.

Sécurité de la Supply Chain :

- Vérification systématique des Digests OCI. Helm v4 s'assure que le binaire déployé n'a pas été altéré entre le registre et le cluster.

Plugins Helm

Qu'est-ce qu'un plugin ? Un outil externe écrit dans n'importe quel langage, intégré nativement à la CLI helm.

Fonctionnement : Helm délègue l'exécution au binaire du plugin en lui passant le contexte (kubeconfig, namespace, etc.).

Nouveauté Helm v4 : Support des plugins WebAssembly (WASM) pour une exécution isolée et sécurisée.

Fin de l'exécution arbitraire de scripts shell non vérifiés par défaut (meilleure posture DevSecOps).

Les Plugins Incontournables

Plugin	Usage principal	Pourquoi l'utiliser ?
helm-diff	Prévisualisation	Affiche un <code>diff</code> coloré entre la release installée et la nouvelle version avant d'appliquer. Indispensable pour éviter les erreurs.
helm-unittest	Qualité / CI	Permet d'écrire des tests unitaires en YAML pour valider que vos templates génèrent le bon code K8S (Tests de conformité).
helm-mapkubeapis	Migration	Met à jour automatiquement les releases Helm lorsque les APIs Kubernetes deviennent obsolètes (ex: passage de <code>v1beta1</code> à <code>v1</code>).

Qu'est-ce que la CNCF ?

Une fondation à but non lucratif (sous l'égide de la Linux Foundation) créée en 2015.

Sa mission : Rendre le "Cloud Native" universel et durable en gérant des projets open source critiques.

Elle rassemble les géants du secteur (Google, AWS, Microsoft, Red Hat) pour **garantir l'interopérabilité**.

Les niveaux de maturité des projets

Sandbox : Projets expérimentaux ou débutants.

Incubating : Projets utilisés en production, répondant à des critères de sécurité et de gouvernance.

Graduated : Le stade ultime (ex: Kubernetes, Prometheus, Helm). C'est le gage de stabilité maximale et de large adoption par l'industrie.

<https://landscape.cncf.io/>

Pourquoi est-ce important pour Helm ?

Indépendance : Helm n'appartient pas à une seule entreprise ; son évolution est dictée par la communauté.

Standardisation : Garantit que Helm fonctionne de la même manière sur GCP, AWS, Azure ou un cluster On-Premise.

Écosystème : Helm est conçu pour s'emboîter parfaitement avec les autres projets "Graduated" (comme Fluentd pour les logs ou Envoy pour le réseau).

Standardisation de Helm

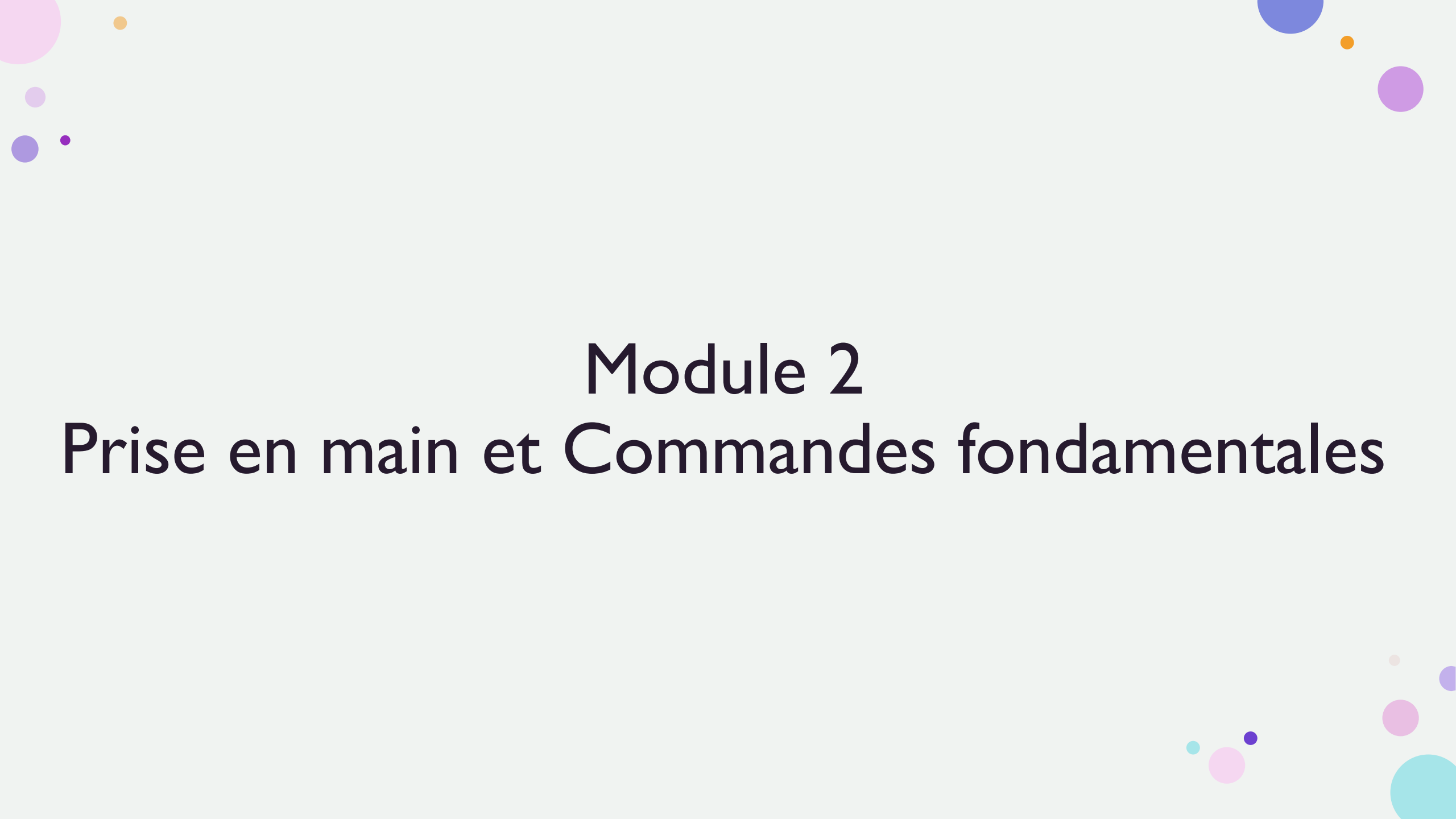
Projet "Graduated" de la CNCF : Niveau de maturité maximal (comme Kubernetes et Prometheus).

Interopérabilité : Supporté par tous les grands fournisseurs Cloud (AWS EKS, Google GKE, Azure AKS).

L'essor de l'OCI (Open Container Initiative) :

- En 2026, on ne sépare plus les images Docker et les Charts Helm.
- On utilise les mêmes registres (Harbor, AWS ECR, GitHub Packages) pour stocker les deux.

Écosystème : Artifact Hub est devenu la place de marché centrale pour trouver des logiciels prêts à l'emploi.

The slide features a light blue background with decorative elements in the corners. The top-left corner has a cluster of pink, orange, and purple circles. The top-right corner has a blue circle, an orange dot, and a purple circle. The bottom-right corner has a cyan circle, a pink circle, a purple dot, and a small orange dot.

Module 2

Prise en main et Commandes fondamentales

Installation et Configuration

Objectif : Préparer l'environnement de travail.

Le binaire helm : Un simple exécutable Go, sans dépendances.

Vérification : *helm version* (S'assurer d'être en v4).

Auto-complétion : L'astuce indispensable pour gagner du temps en CLI.

```
helm completion bash > /etc/bash_completion.d/helm
```

Architecture client : Helm utilise votre contexte kubectl actuel (~/.kube/config).

Rappel : Pas de serveur "Tiller" à installer dans le cluster.

Les Repositories

Artifact Hub : Le portail central (le "Docker Hub" de Helm).

<https://artifacthub.io/>

Gestion des dépôts :

- `helm repo add [nom] [url]` : Ajouter une source (ex: Bitnami).
- `helm repo update` : Synchroniser les index locaux.
- `helm search repo [nom]` : Chercher un package localement.

Focus 2026 : Registres OCI

- Utiliser `helm pull oci://...` pour les registres modernes (GitHub, Azure, Harbor).
- Plus besoin d'un serveur de repo spécifique, on réutilise la stack Docker.

Déployer sa première Release

Objectif : Transformer un Chart en application vivante.

La commande install : *helm install [nom-release] [nom-chart]*

L'importance du Namespace : Par défaut dans default, mais recommandé d'utiliser *--create-namespace -n [mon-ns]*.

Le cycle de vie :

- *helm list* : Voir ce qui tourne (statut deployed, failed).
- *helm status [nom-release]* : Voir les ressources K8s créées et les notes de fin d'installation.

Personnalisation avec les "Values"

Concept : Ne jamais modifier le code du Chart, surcharger les paramètres.

Le fichier values.yaml : Contient les réglages par défaut du créateur.

Méthode 1 : Le fichier externe (Recommandé)

```
helm install -f mes-valeurs.yaml ...
```

Méthode 2 : L'injection directe (Rapide)

```
helm install --set replicaCount=3 ...
```

Priorité : Le dernier argument écrase les précédents. --set a la priorité maximale.

Mise à jour et Rollback

Point fort : La gestion du temps et des versions.

helm upgrade : Appliquer une nouvelle configuration ou une nouvelle version du Chart.

L'historique : *helm history [nom-release]*

Helm garde une trace de chaque modification (Revision 1, 2, 3...).

Le Rollback atomique :

helm rollback [nom-release] [numero-revision]

Kubernetes se charge de remettre l'ancien état (Pods, ConfigMaps) instantanément.

Nettoyage et Inspection

Objectif : Garder un cluster propre.

Désinstallation : *helm uninstall [nom-release]*

Supprime toutes les ressources liées (Services, Deployments, Secrets).

Inspection sans installer :

helm show values [chart] : Voir ce qu'on peut configurer.

helm template [chart] : Afficher le YAML final généré dans le terminal (pour debug).

Lab I - Prise en Main

Il vous est demandé de suivre les instructions du fichier `labs/helm/lab1.md`

Ce premier lab a pour but de vous familiariser avec l'écosystème Helm avant de toucher à notre application DevOps-News.

Vous allez installer Helm, explorer Artifact Hub, déployer un chart communautaire et manipuler le cycle de vie d'une release (install, upgrade, rollback, uninstall).

The slide features a light gray background with decorative elements in the corners. The top-left corner contains a cluster of pink, purple, and orange circles. The top-right corner has blue, orange, and purple circles. The bottom-right corner is decorated with pink, purple, and teal circles.

Module 3

Personnalisation des déploiements

Le fichier values.yaml, source de vérité

Principe : Helm sépare le code (templates) de la donnée (configurations).

Le fichier par défaut : Chaque Chart possède un values.yaml à sa racine. Il sert de documentation vivante et définit les valeurs par défaut.

La structure : Généralement organisé par blocs logiques (image, service, ingress, resources).

Le principe de "Merge" (Fusion) : Helm ne remplace pas tout le fichier, il fusionne vos modifications avec les valeurs par défaut.

Bonne pratique : Ne jamais modifier le values.yaml d'un Chart tiers (ex: Bitnami). Créez toujours votre propre fichier my-values.yaml.

Stratégies de surcharge (Overriding)

Objectif : Adapter le déploiement au contexte (Dev vs Prod).

Multiple fichiers de valeurs :

- *helm install my-app ./chart -f values.yaml -f values-prod.yaml*
- Note : L'ordre compte ! Le dernier fichier mentionné écrase les clés communes des précédents.

Surcharge ciblée via --set :

- Utile pour les paramètres dynamiques (ex: tag d'image issu d'une CI/CD).
- Syntaxe pour les objets complexes : *--set ingress.hosts[0].host=api.exemple.com*.

Surcharge de fichiers : Utiliser *--set-file* pour injecter le contenu d'un certificat ou d'une clé SSH sans le copier en texte clair.

Vers un déploiement intelligent

Requests & Limits (Fine-tuning) :

- Ne jamais coder en dur dans les templates ce qui peut varier entre environnements ou utilisateurs.
- Méthode : Utiliser la fonction toYaml pour injecter directement un bloc depuis le values.yaml.
- Exemple : resources: {{ toYaml .Values.resources | nindent 10 }}.

HPA (Horizontal Pod Autoscaler) :

- Logique conditionnelle : Utiliser un bloc {{ if .Values.autoscaling.enabled }} pour ne générer l'objet HorizontalPodAutoscaler que si nécessaire.
- Lier dynamiquement le minReplicas et maxReplicas aux valeurs globales du Chart.

Placement des Pods (Affinité & Tolerations) :

- nodeSelector : Pour les contraintes simples (ex: disktype: ssd).
- Affinity & Anti-Affinity : Indispensable pour la haute disponibilité (éviter que deux réplicas d'un même service ne tombent sur le même nœud physique).
- Taints & Tolerations : Pour permettre aux Pods de "tolérer" des nœuds réservés (ex: nœuds GPU).

Exposition flexible et sécurisée (1 / 2)

Abstraction du Service (L4) :

- Ne pas fixer le type. Utiliser une variable : `type: {{ .Values.service.type | default "ClusterIP" }}`.
- Expert : Gérer les `externalTrafficPolicy`: Local via les values pour conserver l'IP source du client (essentiel pour les logs et la sécurité).

Ingress & Routage (L7) :

- Logique de présence : Utiliser `{{ if .Values.ingress.enabled }}` pour ne générer l'objet que si nécessaire.
- Multi-Hôtes : Boucler avec `{{ range .Values.ingress.hosts }}` pour gérer plusieurs domaines (FQDN) dynamiquement.

Exposition flexible et sécurisée (2 / 2)

Sécurité TLS & Cert-Manager :

- Automatiser le SSL en passant l'annotation cert-manager.io/cluster-issuer via les values.
- Permettre la configuration dynamique du bloc tls: dans le template.

Annotations Dynamiques (Multi-Cloud) :

- Utiliser la fonction toYaml pour injecter des annotations spécifiques au fournisseur (ex: alb.ingress.kubernetes.io/scheme: internet-facing pour AWS).
- Avantage : Le même Chart devient portable d'un Cloud à l'autre simplement en changeant le fichier de valeurs.

Debugging et Dry-Run

Objectif : Vérifier avant d'appliquer.

La commande lint : `helm lint ./my-chart`

Vérifie la syntaxe et les bonnes pratiques du Chart.

Le mode "Dry-Run" : `helm install --dry-run --debug ...`

Simule l'installation et affiche le manifest YAML résultant sans rien créer sur le cluster.

Diffing (Plugin helm-diff) :

- Très utilisé aujourd'hui : permet de voir exactement quelles lignes du YAML vont changer avant de lancer un upgrade.

Le Post-Rendering (I / 2)

Le Problème :

- Vous utilisez un Chart communautaire (ex: Prometheus, Keycloak).
- Un paramètre spécifique (annotation de sécurité, sidecar, label propriétaire) n'est pas exposé dans le values.yaml.
- Risque : Devoir "forker" le code, perdre les mises à jour et porter une dette technique.

La Solution : Le pipeline de rendu (Helm + Kustomize)

- Helm produit le manifest YAML complet.
- Au lieu de l'envoyer au cluster, il le transmet via STDIN à un binaire externe (le post-renderer).
- Outil de référence : Kustomize est le partenaire idéal pour "patcher" le résultat à la volée.

Le Post-Rendering (2 / 2)

Nouveauté Helm v4 & Sécurité :

- Les post-renderers peuvent désormais être encapsulés dans des plugins WASM (WebAssembly).
- Avantage : Isolation complète du script de modification, garantissant qu'il ne peut pas compromettre la machine qui exécute la commande Helm.

Le Wrapper Script : Un petit script `kustomize-wrapper.sh` est souvent nécessaire pour que Helm puisse parler à Kustomize (car Helm envoie tout le YAML dans un seul flux, alors que Kustomize attend souvent des fichiers séparés).

L'alternative "GitOps" : Avec un outil comme ArgoCD, le post-rendering est géré nativement via les multiple plugin types.

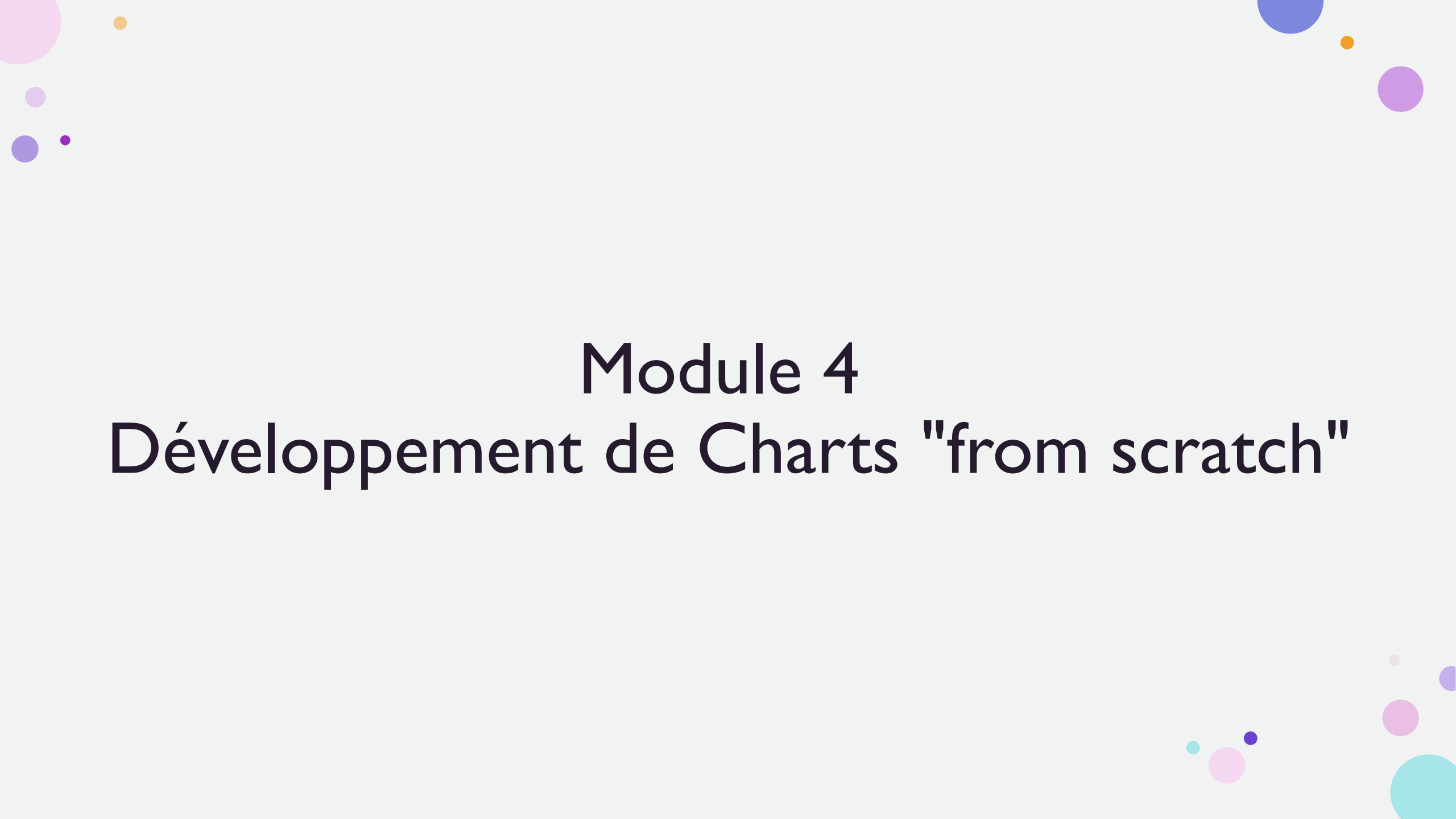
Mise en garde : Si on modifie 80% du Chart via Kustomize, c'est qu'il est temps de créer son propre Chart "From Scratch".

Lab 2 - Personnalisation

Il vous est demandé de suivre les instructions du fichier `labs/helm/lab2.md`

Dans ce lab, nous allons déployer notre application DevOps-News à l'aide d'un chart communautaire et expérimenter toutes les stratégies de personnalisation de Helm :

- Fichiers de valeurs,
- Surcharge ciblée,
- Multi-environnements Dev/Prod.

The slide features a light gray background with decorative elements in the corners. The top-left corner contains a cluster of circles in shades of pink, purple, and orange. The top-right corner has circles in blue, orange, and purple. The bottom-right corner is decorated with circles in teal, pink, and purple.

Module 4

Développement de Charts "from scratch"

Initialisation d'un Chart

Objectif : Partir d'une structure saine et standardisée.

La commande magique : *helm create [nom-du-chart]*

Ce que génère Helm : Une structure "Boilerplate" (modèle) complète.

- Un exemple de Deployment, Service, Ingress et ServiceAccount.
- Un fichier values.yaml pré-rempli.
- Un dossier de tests.

Pourquoi nettoyer ? Le boilerplate est très complet (trop parfois). En production, on épure souvent pour ne garder que l'essentiel.

Anatomie et Arborescence

Chart.yaml : Les métadonnées (Nom, version de l'app, version du package).

values.yaml : Les paramètres par défaut (la configuration).

templates/ : Les fichiers YAML , fichiers à trous, qui seront remplis par le moteur Go.

charts/ : Dossier pour les dépendances (Sub-charts).

.helmignore : Pour exclure des fichiers du packaging (comme .gitignore).

Le fichier Chart.yaml

Le type de Chart :

- application : Déploie des ressources (Pods, Services).
- library : Fournit des fonctions de template réutilisables (sans déploiement direct).
- Nouveau en v4 - plugin : Permet désormais de packager des extensions Helm (WASM) directement sous forme de Charts pour une distribution simplifiée.

Le triple versioning :

- version : Version du package (SemVer).
- appVersion : Version de l'application (Image).
- digest (Optionnel mais recommandé) : En v4, on peut spécifier le hash immuable (SHA256) du chart pour garantir la sécurité de la Supply Chain.

Dépendances (Refonte interne) :

- Toujours déclarées ici, mais leur résolution est désormais parallélisée en v4.
- Support natif renforcé pour les registres OCI (Docker Registry, GHCR, etc.) comme source de dépendance principale, marquant la fin progressive des serveurs de dépôts traditionnels (index.yaml).

Les Objets Globaux (Le contexte)

Lors du rendu, Helm met à disposition plusieurs objets racines, dont la description est donnée ci-dessous

.Values : Tout ce qui provient du fichier values.yaml.

.Release : Infos sur l'instance (Nom de la release, namespace, révision).

.Chart : Contenu du fichier Chart.yaml.

.Capabilities : Infos sur le cluster cible (Version de Kubernetes, APIs supportées).

.Template : Infos sur le fichier de template en cours de traitement.

Syntaxe de base d'un Template

Objectif : Remplacer les valeurs statiques par du dynamique.

Les accolades : {{ .Values.mavariabile }}.

Exemple concret (Deployment) ci-contre

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-deploy
spec:
  replicas: {{ .Values.replicaCount }}
```

La sécurité par défaut : Utiliser des fonctions comme *quote* pour éviter les erreurs YAML sur les chaînes de caractères spéciales.

Packaging et Distribution

Objectif : Créer l'archive prête à l'emploi.

helm package ./mon-chart : Génère un fichier .tgz.

Vérification du package : `helm inspect chart ./mon-chart-0.1.0.tgz`.

Provenances : Signature des charts pour garantir l'origine (fichiers .prov).

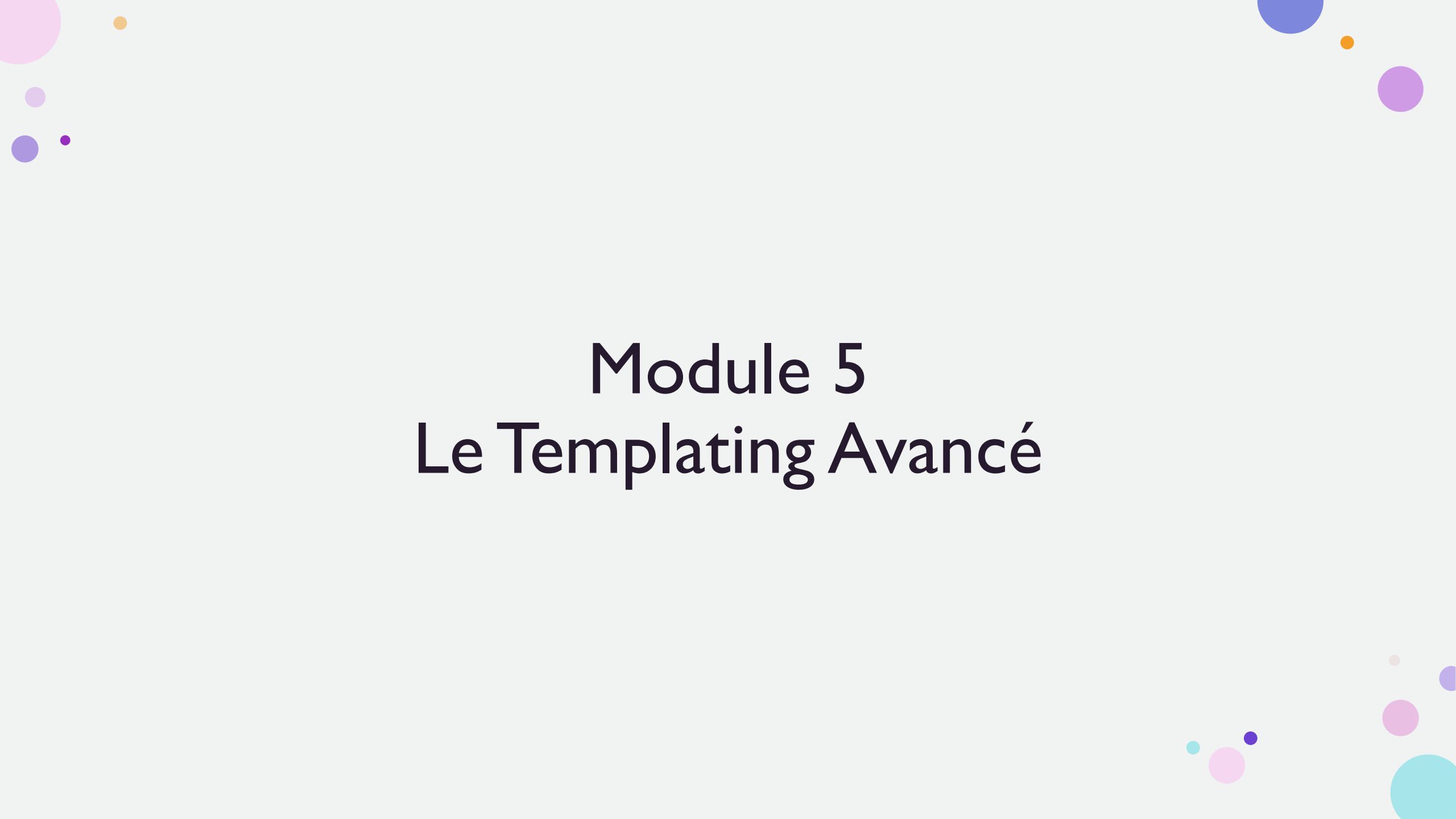
Publication : Envoyer le .tgz vers un registre OCI ou un repo Git.

Lab 3 - Chart from scratch

Il vous est demandé de suivre les instructions du fichier `labs/helm/lab3.md`

C'est le lab central de la formation. Vous allez créer de zéro le chart Helm complet de l'application DevOps-News, en packager une version et la déployer sur GKE.

Ce lab est autonome : les consignes décrivent ce que le chart doit produire, mais c'est à vous d'écrire les templates.

The slide features a light gray background with decorative elements in the corners. The top-left corner contains a cluster of circles in shades of pink, purple, and orange. The top-right corner has circles in blue, orange, and purple. The bottom-right corner is decorated with circles in teal, pink, and purple. The main text is centered in a dark purple font.

Module 5

Le Templating Avancé

Introduction à la Logique de Rendu

Helm n'est pas un simple moteur de recherche/remplacement.

Le moteur Go Templates : Puissant, il permet d'injecter de la logique (Boucles, Conditions).

La bibliothèque Sprig : Helm inclut plus de 100 fonctions utilitaires (manipulation de chaînes, encodage Base64, calculs mathématiques). <https://masterminds.github.io/sprig/>

Le flux de données :

1. Lecture du values.yaml.
2. Exécution de la logique Go dans les fichiers de templates/.
3. Production d'un manifeste YAML final envoyé au cluster.

Structures de Contrôle (If / Else)

Cas d'usage : Rendre des blocs optionnels (ex: Activer un Ingress ou des Sidecars).

L'importance des tirets (-) :

- `{{- : Supprime les espaces/sauts de ligne avant le bloc.`
- `-}} : Supprime les espaces/sauts de ligne après.`
- Sans les tirets, votre YAML final contiendra des lignes vides qui peuvent fausser l'indentation.

```
{{- if .Values.ingress.enabled }}  
apiVersion: networking.k8s.io/v1  
kind: Ingress  
...  
{{- else }}  
# Ingress désactivé  
{{- end }}
```

Itérations avec range (Boucles)

Cas d'usage : Générer des listes de variables d'environnement, de volumes ou de ports.

Attention au Scope : À l'intérieur du range, le point . devient l'élément de la liste. Pour accéder à la racine, utilisez \$.Values.

```
env:
  {{- range $.Values.extraEnv }}
  - name: {{ .name }}
    value: {{ .value | quote }}
  {{- end }}
```

```
data:
  {{- range $key, $val := $.Values.configData }}
  {{ $key }}: {{ $val | quote }}
  {{- end }}
```

Manipulation du Scope avec with

Objectif : Raccourcir les chemins d'accès aux variables.

Au lieu d'écrire partout `.Values.persistence.storage.class`, on change le scope localement

Règle d'or : Tout comme avec `range`, le point racine `.` est "perdu" à l'intérieur du bloc.

```
{{- with .Values.persistence }}  
storage: {{ .storage }}  
class: {{ .class }}  
{{- end }}
```

Fonctions, Pipelines et Indentation

Objectif : Transformer la donnée à la volée.

Les Pipelines (|) : Enchaîner les transformations comme en Linux.

```
{{ .Values.app.name | upper | quote }}
```

La fonction nindent (Cruciale) :

Elle ajoute un saut de ligne (n) et X espaces.

Indispensable pour injecter des blocs YAML entiers :

```
resources:  
  {{- toYaml .Values.resources | nindent 10 }}
```


Templates nommés et Helpers (`_helpers.tpl`)

Objectif : Factoriser le code pour éviter la répétition (DRY : Don't Repeat Yourself).

Définition : Dans `_helpers.tpl`, on définit des blocs réutilisables avec `define`.

Inclusion : On les appelle avec `include`.

Pourquoi `include` plutôt que `template` ?

- *include* permet de passer le résultat à une autre fonction (ex: l'indenter).
- *template* est une action native Go plus limitée.

Accès aux fichiers externes (.Files)

Cas d'usage : Injecter une configuration complexe ou un certificat sans polluer le values.yaml.

Récupérer un fichier : `{{ .Files.Get "config/nginx.conf" }}`

Exemple : Injecter dans une ConfigMap

Sécurité : .Files ne permet pas d'accéder à des fichiers en dehors du dossier du Chart.

```
data:  
  server.conf: |-  
    {{ .Files.Get "server.conf" | nindent 6 }}
```

Lab 4 - Templating avancé

Il vous est demandé de suivre les instructions du fichier `labs/helm/lab4.md`

Dans ce lab, vous allez enrichir le chart `devops-news` créé au Lab 3 avec les techniques de templating avancé :

- Helpers réutilisables (`_helpers.tpl`),
- Conditions,
- Boucles *range*,
- Manipulation du scope avec *with*,
- Pipelines de fonctions
- Accès aux fichiers externes.

The slide features a light gray background with decorative elements in the corners. The top-left corner contains a cluster of pink, purple, and orange circles. The top-right corner has blue, purple, and orange circles. The bottom-right corner is decorated with pink, purple, and teal circles. The main title is centered in a dark purple font.

Module 6

Sécurité, Qualité et Tests

Valider la conformité avec helm lint

Objectif : Détecter les erreurs avant le déploiement.

La commande *helm lint* : Analyse statique du code.

- Vérifie la structure des dossiers.
- Vérifie la validité du YAML après rendu.
- Alerte sur les bonnes pratiques manquantes (ex: icon manquante dans Chart.yaml).

Aller plus loin avec *Kube-linter* ou *Datree* :

- Vérifier si les images utilisent le tag latest (interdit en prod).
- Vérifier l'absence de privilèges root dans les containers.

Tester les déploiements avec helm test

Objectif : Valider que l'application fonctionne une fois installée.

Le dossier templates/tests/ : Contient des Pods de test (souvent basés sur curl ou wget).

L'annotation @helm.sh/hook: test : Indique à Helm que ce Pod ne doit être lancé que lors d'un test.

Exécution : *helm test [release]*

- Helm lance le Pod, attend un code de sortie 0.
- Si le test échoue, la release peut être considérée comme instable.

Le défi des Secrets dans Helm

Problématique : Pourquoi le YAML natif ne suffit pas ?

Le danger du texte clair : Ne jamais stocker de mots de passe ou de clés API dans values.yaml (risque de commit sur Git).

Le fonctionnement natif : Helm crée des objets Secret Kubernetes, mais ils ne sont que encodés en Base64 (pas chiffrés).

Les solutions du marché :

- External Secrets Operator : Récupérer les secrets depuis un coffre-fort (Vault, AWS Secrets Manager).
- Helm Secrets (Plugin) : Chiffrer les fichiers de valeurs localement avec Sops.
- Bitnami Sealed Secrets : Transformer des secrets en objets chiffrés que seul le cluster peut lire.

Les Helm Hooks (Cycle de vie avancé)

Cas d'usage : Actions automatiques avant/après installation.

Objectif: Intervenir à des moments précis du cycle de vie.

Hooks courants :

- pre-install / pre-upgrade : Lancer une sauvegarde ou une migration de base de données.
- post-install : Envoyer une notification Slack.
- pre-delete : Nettoyer des ressources externes (ex: un bucket S3).

Gestion des poids (weight) : Ordonner l'exécution de plusieurs hooks.

Signature et Provenance des Charts

Objectif: Sécuriser de la chaîne d'approvisionnement (Supply Chain).

Pourquoi signer ? Garantir que le Chart n'a pas été modifié entre le développeur et le cluster.

L'outil GnuPG : Création d'une clé de signature.

Le fichier de provenance (.prov) : Généré lors du `helm package --sign`.

Vérification à l'installation : `helm install --verify ...`

Helm refuse l'installation si la signature ne correspond pas à la clé publique de confiance.

Lab 5 - Qualité & Sécurité

Il vous est demandé de suivre les instructions du fichier `labs/helm/lab5.md`

Un chart qui fonctionne n'est pas forcément un chart de qualité.

Dans ce lab, vous allez enrichir le chart `devops-news` avec les mécanismes de qualité et de sécurité indispensables en production :

- Validation par schéma,
- Fichier `NOTES.txt`,
- Tests Helm,
- Hooks de cycle de vie
- Gestion sécurisée des secrets.

Lab 6 - Tests Unitaires

Il vous est demandé de suivre les instructions du fichier `labs/helm/lab6.md`

Un chart Helm sans tests est un chart non maintenable.

Dans ce lab, vous allez écrire une suite de tests unitaires complète pour le chart `devops-news` : tester les cas nominaux, les cas limites, les conditions (if) et les boucles (range).

Ces tests s'intègrent ensuite dans un pipeline CI/CD pour garantir que chaque modification du chart ne casse pas le comportement attendu.

The slide features a light gray background with decorative elements in the corners. The top-left corner has a cluster of pink, purple, and orange circles. The top-right corner has blue, orange, and purple circles. The bottom-right corner has a cluster of pink, purple, and teal circles.

Module 7

Intégration avec Kustomize

Présentation

Principe : Utiliser Helm pour générer le YAML de base, puis passer automatiquement ce résultat à Kustomize pour appliquer des modifications de dernière minute.

Pourquoi faire ?

- Injecter des annotations de sécurité spécifiques à votre entreprise.
- Modifier une ressource qu'un Chart tiers ne permet pas de paramétrer.
- Garder vos modifications séparées du code source du Chart.

Exemple de commande :

```
helm install ma-release ./mon-chart --post-renderer ./kustomize-wrapper.sh
```

Des outils comme Argo CD permettent de faire cela nativement dans l'interface (section kustomize au sein d'une source helm).

Différences Helm / Kustomize

Critère	Kustomize	Helm
Approche	Patch déclaratif (merge/overlay)	Templating Go avec values.yaml
Courbe d'apprentissage	Faible — YAML pur	Modérée — Go templates
Logique conditionnelle	Limitée (patches ciblés)	Oui — if/range/with
Packaging / OCI	Non	Oui — chart registry
Intégration kubectl	Native (v1.14+)	CLI séparé (helm)
Gestion des secrets	secretGenerator + external tools	Helm Secrets / vault plugin
GitOps (Argo/Flux)	Natif	Natif
Tests / hooks	Limités	Complets (helm test, hooks)

Lab 7 - Intégration Kustomize

Il vous est demandé de suivre les instructions du lab `helm/lab7.md`

Dans ce lab, vous allez explorer deux scénarios complémentaires de collaboration entre Helm et Kustomize :

- Le post-rendering (patcher la sortie d'un chart tiers en temps réel)
- L'approche hybride (Helm génère la base, Kustomize gère les overlays d'environnement).

The slide features a light gray background with decorative elements in the corners. The top-left corner has a cluster of pink, purple, and orange circles. The top-right corner has blue, orange, and purple circles. The bottom-right corner has a cluster of pink, purple, and teal circles.

Module 8

GitOps avec Argo CD et Helm

Pourquoi automatiser Helm ?

Problématique : Éviter le "Snowflake Cluster" (cluster configuré manuellement).

Traçabilité : Qui a déployé quoi, quand et avec quelles valeurs ?

Reproductibilité : Être capable de reconstruire tout un environnement à partir d'un dépôt Git.

Séparation des responsabilités : Le développeur pousse du code, la CI/CD gère le déploiement Helm.

"Single Source of Truth" : Le dépôt Git contient l'état désiré du cluster.

Helm dans une Pipeline CI/CD classique

Workflow : Jenkins, GitLab CI, GitHub Actions.

1 - Build & Lint : Vérifier la syntaxe du Chart (helm lint).

2 - Test : Lancer des tests unitaires sur les templates (outil helm-unittest).

3 - Push : Packager le Chart et l'envoyer vers un registre OCI (Harbor, AWS ECR).

4 - Deploy : Exécuter helm upgrade --install vers le cluster cible.

Problème : Si le cluster dérive manuellement (drift), la CI classique ne le voit pas.

Le concept de GitOps

Principe : Utiliser un dépôt Git comme "Source Unique de Vérité" pour l'infrastructure et les applications.

Le Pull Model vs Push Model :

- *CI/CD Classique (Push)* : La pipeline "pousse" les changements vers K8s. Problème : Si quelqu'un modifie le cluster manuellement, la CI ne le sait pas.
- *GitOps (Pull)* : Un agent (Argo CD) surveille Git et "tire" la configuration vers le cluster.

Réconciliation automatique (Self-healing) : Si l'état du cluster dérive (Drift), Argo CD écrase les modifications manuelles pour revenir à ce qui est défini dans Git.

L'approche GitOps

Principe : Inverser le flux de déploiement.

Un agent dans le cluster surveille Git et synchronise l'état.

Argo CD et Helm :

- Argo CD sait lire un Chart Helm directement depuis un repo Git ou un registre OCI.
- Il permet de surcharger les values.yaml via son interface ou son manifeste Application.

Avantages : Auto-healing (si quelqu'un modifie un Service manuellement, Argo CD le remet selon le Chart Helm).

ArgoCD - Motivations

Le cluster comme cible mouvante

Dans un workflow CI/CD classique, la pipeline pousse les changements vers Kubernetes. Mais si un opérateur modifie directement une ressource avec kubectl, le cluster dérive silencieusement de ce que Git décrit. Personne ne le sait, jusqu'à la prochaine panne.

Le problème de traçabilité

Qui a déployé quoi, quand, et avec quelles valeurs ? Sans GitOps, la réponse est dans les logs de la CI/CD — s'ils n'ont pas expiré. Avec GitOps, chaque état du cluster correspond à un commit Git signé : la traçabilité est totale et permanente.

La reproductibilité impossible

Reconstruire un environnement de production après un incident majeur est un exercice périlleux si la configuration est éparpillée entre des pipelines, des kubectl apply manuels et des valeurs Helm oubliées. Le GitOps garantit qu'un seul dépôt Git suffit à recréer l'environnement à l'identique.

ArgoCD - Motivations

La sécurité du cluster renforcée

Dans un modèle CI/CD Push, les pipelines GitHub Actions ou GitLab CI ont besoin de droits d'écriture (cluster-admin) sur le cluster. Avec Argo CD, seul l'agent dans le cluster a ces droits — la pipeline ne fait que pousser du code dans Git. La surface d'attaque est radicalement réduite.

L'auto-réparation (Self-healing)

Argo CD surveille en permanence l'état réel du cluster et le compare à l'état décrit dans Git. Si quelqu'un modifie manuellement un Deployment ou supprime accidentellement un Service, Argo CD détecte l'écart et remet automatiquement le cluster dans l'état attendu, sans intervention humaine.

L'union naturelle avec Helm

Argo CD comprend nativement le langage Helm : il lit un chart, applique les fichiers de valeurs par environnement, et synchronise le résultat. Le rollback d'une application ne passe plus par helm rollback — il suffit d'un git revert pour qu'Argo CD revienne à la version précédente automatiquement. Git devient la télécommande du cluster.

Argo CD et les Helm Charts

Argo CD possède un "support natif" pour Helm. Il n'a pas besoin de helm sur votre poste, il embarque sa propre logique de rendu.

Trois sources possibles pour Argo CD :

- Git Repository : Le Chart est stocké dans vos dossiers Git.
- Helm Repository : Argo pointe vers un repo HTTPS (ex: Bitnami).
- OCI Registry (2026) : Argo récupère le Chart directement depuis un registre Docker (Harbor, ECR).

Surcharge des valeurs : On définit les values directement dans l'objet Application d'Argo CD, ce qui remplace le besoin de passer des fichiers -f values.yaml manuellement.

Anatomie d'une "Application" Argo CD

Pour déployer avec Argo CD, on crée une ressource Kubernetes de type Application. Les différents champs à renseigner sont décrits ci-dessous.

source : Où se trouve le Chart ? (Repo URL, Target Revision, Chart Name).

destination : Vers quel cluster et quel namespace déployer ?

helm configuration : values à appliquer

syncPolicy : * **Automated** : Déploiement dès qu'un commit est détecté.

Prune : Supprimer les ressources K8s si elles sont supprimées du Chart.

SelfHeal : Corriger les modifications manuelles interdites.

Avantages de l'union Helm + Argo CD

Visibilité Graphique : Argo CD offre une interface web pour visualiser toutes les ressources créées par un Chart Helm (Pods, Services, Ingress) et leur état de santé.

Gestion multi-clusters : Déployer le même Chart Helm sur 10 clusters différents avec des valeurs spécifiques depuis une seule interface.

Rollback simplifié : On ne fait plus helm rollback. On fait un git revert sur le commit précédent, et Argo CD s'occupe de remettre l'ancienne version du Chart.

Sécurité : Plus besoin de donner des accès admin aux pipelines CI (GitHub Actions/GitLab CI). Seul Argo CD possède les droits d'écriture sur le cluster.

Gestion des secrets en GitOps

Défi : Comment stocker les mots de passe dans Git pour Helm ?

Sops (Secrets Operations) : Chiffrer uniquement les valeurs sensibles dans un fichier secrets.yaml.

Sealed Secrets : Transformer les secrets en ressources K8s "scellées" que l'on peut versionner sans risque.

External Secrets Operator (Recommandé aujourd'hui) :

- Le Chart Helm définit un ExternalSecret.
- L'opérateur va chercher la vraie valeur dans AWS Secrets Manager ou HashiCorp Vault.
- Helm ne manipule jamais la donnée secrète brute.

Stratégies de dépendances complexes

Objectif : Gérer des applications distribuées.

Les Sub-charts (Dossier charts/) : Inclure une base de données ou un cache comme dépendance de votre application principale.

Le fichier Chart.lock : Verrouiller les versions des dépendances pour garantir que le déploiement est identique partout.

L'Umbrella Chart :

- Un Chart vide de ressources propres, qui ne sert qu'à regrouper 10 autres Charts (Microservices).
- Idéal pour déployer une plateforme complète en une commande.

Lab 8 - GitOps avec ArgoCD

Il vous est demandé de suivre les instructions du fichier `labs/helm/lab8.md`

Dans ce lab final, vous allez passer du déploiement impératif (`helm install`) au déploiement déclaratif et automatisé avec Argo CD.

Vous connecterez votre chart `devops-news` à une instance Argo CD, observerez le mécanisme de self-healing en cas de drift, et gérerez les secrets de manière GitOps-compatible.

The image features a light gray background with several decorative circles of various colors (pink, purple, blue, orange, teal) scattered in the corners. The text "Pour aller plus loin" is centered in a dark purple font.

Pour aller plus loin

Umbrella Chart

Définition: Un umbrella chart (ou chart parent) est un chart Helm qui n'embarque pas directement de ressources Kubernetes, mais qui orchestre plusieurs sous-charts (appelés dépendances) en un seul déploiement cohérent.

Principe

Plutôt que de déployer chaque composant d'une application séparément:

```
helm install frontend ./frontend
```

```
helm install backend ./backend
```

```
helm install redis ./redis
```

```
helm install postgresql ./postgresql
```

Un umbrella chart regroupe tout en une seule commande:

```
helm install my-app ./umbrella -f values-prod.yaml
```

Umbrella Chart - Structure

```
umbrella/  
├── Chart.yaml      # Déclare les dépendances  
├── values.yaml     # Valeurs globales + surcharges par sous-chart  
├── charts/         # Sous-charts téléchargés (helm dependency update)  
│   ├── postgresql/  
│   ├── redis/  
│   └── backend/  
└── templates/     # Optionnel — ressources propres à l'umbrella  
    └── configmap.yaml
```

Umbrella Chart - Chart.yaml

```
apiVersion: v2
name: my-app
version: 1.0.0

dependencies:
  - name: postgresql
    version: "12.5.6"
    repository: "https://charts.bitnami.com/bitnami"
    condition: postgresql.enabled

  - name: redis
    version: "17.3.0"
    repository: "https://charts.bitnami.com/bitnami"
    condition: redis.enabled
```

Le champ **condition** permet d'activer ou désactiver un sous-chart depuis les values.

Umbrella Chart - values.yaml

Chaque sous-chart est accessible via une clé qui porte son nom

```
postgresql:  
  enabled: true  
  auth:  
    username: myuser  
    password: secret  
    database: mydb  
  
  redis:  
    enabled: true  
    architecture: standalone  
  
  backend:  
    enabled: true  
    image: tag: "2.1.0"
```

Umbrella Chart - Valeurs globales

Un umbrella peut définir des valeurs partagées entre tous les sous-charts via la clé global

Dans les sous-charts, ces valeurs sont accessibles via `.Values.global` — c'est le seul mécanisme natif de communication entre sous-charts.

```
global:  
  imageRegistry: "registry.mycorp.com"  
  storageClass: "fast-ssd"  
  environment: "production"
```

Umbrella Chart - Avantages

Déploiement atomique : toute la stack en une commande, un seul helm rollback si problème.

Gestion centralisée des versions : le Chart.yaml de l'umbrella fixe les versions de toutes les dépendances, garantissant la reproductibilité.

Surcharge fine : possibilité de tweaker n'importe quel paramètre de n'importe quel sous-chart depuis un seul values.yaml.

Environnements multiples : un values-prod.yaml et un values-dev.yaml suffisent pour piloter toute la stack.

Umbrella Chart - Limites

Couplage fort : tous les composants partagent le même cycle de release. Si vous voulez upgrader uniquement le backend, vous devez quand même passer par l'umbrella entier.

Debugging complexe : un helm template produit des centaines de lignes issues de multiples sous-charts, difficile à lire.

Conflits de nommage : si deux sous-charts définissent des ressources avec le même nom, des collisions peuvent survenir.

Valeurs profondes : surcharger `postgresql.primary.persistence.size` depuis le CLI devient vite verbeux



Helmfile

Le problème : helm install devient fastidieux quand on a 20 apps.

La solution Helmfile : Un fichier helmfile.yaml qui liste toutes les releases, leurs versions, et leurs dépendances.

Fonctionnalités :

1. Gestion de l'ordre de déploiement.
2. Support des variables d'environnement.
3. Intégration parfaite avec les pipelines de déploiement global.



Helmfile : Structure

repositories:

- **name**: bitnami
 - url**: <https://charts.bitnami.com/bitnami>

environments:

dev:

values:

- values/dev.yaml

prod:

values:

- values/prod.yaml

releases:

- **name**: postgresql
 - chart**: bitnami/postgresql
 - version**: "12.5.6"
 - values**:
 - values/postgresql.yaml
 - condition**: postgresql.enabled

Apports de Helmfile

Gestion des dépendances entre releases

Le champ `needs` garantit l'ordre de déploiement — backend ne se déploie qu'après postgresql et redis. L'umbrella chart n'a pas ce mécanisme natif.

Gestion native des environnements

- *helmfile -e prod apply*
- *helmfile -e dev apply*

Helmfile switch automatiquement les valeurs selon l'environnement — plus propre que jongler avec plusieurs `-f values-xxx.yaml`.

Apports de Helmfile

Déploiement sélectif

- *helmfile -l name=backend apply # déploie uniquement le backend*
- *helmfile -l name=backend diff # diff uniquement sur le backend*

Impossible avec un umbrella — c'est tout ou rien.

Diff global

- `helmfile diff`

Affiche les différences de toutes les releases en une commande — très utile en CI/CD avant d'appliquer.

Helmfile vs Umbrella Chart

	Umbrella Chart	Helmfile
Nature	Chart Helm natif	Outil externe
Releases	Une seule	N releases indépendantes
Rollback	Global uniquement	Par release
Ordre de déploiement	Non géré	needs
Gestion d'environnements	Manuel (-f values-prod.yaml)	Natif (-e prod)
Déploiement sélectif	Non	Oui (-l name=xxx)
Packaging / distribution	Oui (OCI, Artifact Hub)	Non
Courbe d'apprentissage	Faible (basé sur Helm)	Modérée
GitOps (Argo/Flux)	Natif	Support partiel

Helmfile ou Umbrella Chart ?

Umbrella chart si :

Vous voulez distribuer votre stack comme un package réutilisable

Les composants sont fortement couplés et versionnés ensemble

Vous êtes dans un contexte GitOps avec Argo CD ou Flux

Helmfile si :

Vous gérez plusieurs applications indépendantes sur un même cluster

Vous avez besoin d'un ordre de déploiement précis entre releases

Vous voulez un diff global et un déploiement sélectif en CI/CD

Vos équipes ont des cycles de release différents par composant

Conclusion

Ce que vous avez construit

Étape	Ce que vous avez fait	Bénéfice
Prise en main	Install, upgrade, rollback sur un chart communautaire	Maîtrise du cycle de vie
Personnalisation	<code>values-dev.yaml</code> / <code>values-prod.yaml</code>	Multi-env sans duplication
Chart from scratch	Templates, Secret, StatefulSet, CronJob	Packaging réutilisable
Templating avancé	Helpers, <code>range</code> , <code>with</code> , <code>toYaml</code>	Code DRY et maintenable
Qualité & Sécurité	Schéma JSON, tests, hooks, secrets	Production-ready
Helm + Kustomize	Post-rendering, overlays hybrides	Flexibilité maximale
GitOps / Argo CD	Self-healing, drift detection	Le cluster réconcilie Git

Les 5 règles d'or à retenir

 Ne jamais committer un secret en clair — External Secrets ou Sealed Secrets, toujours

 Un chart, plusieurs releases — nommer les releases par environnement, pas le chart

 Tags d'image précis, jamais latest — l'immuabilité est une garantie, pas une contrainte

 values.schema.json obligatoire sur les charts distribués — valider avant de déployer

 En GitOps, le rollback c'est git revert — helm rollback devient une exception

La suite logique

Court terme — consolider

- Écrire des tests unitaires sur vos templates avec helm-unittest
- Intégrer helm lint + helm-diff dans votre pipeline CI/CD
- Publier votre chart sur un registre OCI (GitHub Packages, Harbor)

Moyen terme — industrialiser

- Helmfile — orchestrer 10+ releases en un seul fichier déclaratif
- Argo CD ApplicationSets — déployer le même chart sur N clusters automatiquement
- Umbrella Chart — une commande pour déployer toute une plateforme

Long terme — aller plus loin

- Crossplane — provisionner l'infrastructure cloud (GCS, Cloud SQL) via des CRDs Kubernetes
- Flux CD — alternative à Argo CD, plus légère, orientée CLI